

数据结构基础面试题

本文档由公众号：**嵌入式校招菌** 原创整理，欢迎关注获取更多嵌入式校招信息

📌 **嵌入式校招excel** 全年更新中，实习/秋招/春招都包含，纯人工维护，已支持24/25/26届同学毕业，减少招聘信息差，你没听过的公司只要有嵌入式岗位我都会收集，一次付费永久有效，更有嵌入式微信/飞书交流群；用过的同学都说好！

需要的可以关注公众号：**嵌入式校招菌**，对话框回复：**加群**

精选 155 道数据结构与算法高频面试题（C 语言实现），涵盖数组、链表、栈、队列、树、堆、哈希表、图、排序、查找等。
每题配详细答案、C 代码实现和复杂度分析。

★ 题目分类导航

- 一、数组与字符串（Q1~Q20）
- 二、链表（Q21~Q48）
- 三、栈与队列（Q49~Q64）
- 四、树与堆（Q65~Q85）
- 五、哈希表（Q86~Q91）
- 六、排序算法（Q92~Q102）
- 七、图与高级算法（Q103~Q120）

一、数组与字符串（Q1~Q20）

Q1: 数组的基本特性？

💡 **秒懂：** 数组就像一排储物柜——编号连续、大小相同，知道编号就能直接打开（ $O(1)$ 随机访问），但中间插入要把后面的全部往后挪。

- 连续内存，随机访问 $O(1)$
- 插入/删除 $O(n)$ （需要移动元素）
- 大小固定（静态数组）

```
int arr[10];           /* 栈上，编译时确定大小 */
int *p = malloc(n * sizeof(int)); /* 堆上，运行时确定 */
```

Q2: 反转数组？

💡 **秒懂：** 反转数组就像把一排人前后对调——用双指针从两头往中间走，左右交换，走到中间就完成了，时间 $O(n)$ 空间 $O(1)$ 。

使用双指针或递归方式实现：

```

void reverse(int *arr, int n) {
    int left = 0, right = n - 1;
    while (left < right) {
        int tmp = arr[left];
        arr[left] = arr[right];
        arr[right] = tmp;
        left++;
        right--;
    }
}
/* 时间 O(n), 空间 O(1) */

```

Q3: 两数之和(数组已排序)?

 **秒懂：** 有序数组两数之和用双指针——一个从头一个从尾，和太大右指针左移，和太小左指针右移，比暴力两层循环从 $O(n^2)$ 降到 $O(n)$ 。

利用双指针法在已排序数组中寻找目标和：

```

/* 双指针法 */
void two_sum(int *arr, int n, int target) {
    int l = 0, r = n - 1;
    while (l < r) {
        int sum = arr[l] + arr[r];
        if (sum == target) {
            printf("(%d, %d)\n", arr[l], arr[r]);
            return;
        } else if (sum < target) {
            l++;
        } else {
            r--;
        }
    }
}
/* O(n) */

```

Q4: 删除有序数组中的重复元素(原地)?

 **秒懂：** 原地去重就像整理书架——用快慢指针，慢指针指向不重复序列的末尾，快指针往前探路，遇到新值就放到慢指针后面。

使用快慢指针原地去除重复元素：

```
int removeDuplicates(int *arr, int n) {
    if (n <= 1) return n;
    int slow = 0;
    for (int fast = 1; fast < n; fast++) {
        if (arr[fast] != arr[slow]) {
            arr[++slow] = arr[fast];
        }
    }
    return slow + 1; /* 新长度 */
}
/* 快慢指针, O(n) */
```

Q5: 字符串反转?

 **秒懂：** 字符串反转就像把一排扑克牌倒过来——双指针从两头交换到中间，或者用栈先全部压入再弹出，是最基础的面试热身题。

```
/* 字符串反转? - 示例实现 */
void reverse_str(char *s) {
    int len = strlen(s);
    for (int i = 0, j = len - 1; i < j; i++, j--) {
        char tmp = s[i];
        s[i] = s[j];
        s[j] = tmp;
    }
}
```

Q6: 判断回文字符串?

 **秒懂：** 回文判断就像照镜子——从两头向中间比较，对称位置字符相同就是回文，'abcba'是，'abcda'不是，双指针O(n)搞定。

从两端向中间逐字符比较：

```
/* 判断回文字符串? - 示例实现 */
int is_palindrome(const char *s) {
    int l = 0, r = strlen(s) - 1;
    while (l < r) {
        if (s[l] != s[r]) return 0;
        l++;
        r--;
    }
    return 1;
}
```

Q7: strlen/strcpy/strcmp/strcat 自己实现?

💡 秒懂：自己实现这4个函数是面试经典——关键是注意strlen不含'\0'、strcpy要拷贝'\0'、strcmp逐字符比较返回差值、strcat找到末尾再拷贝。

手动实现标准库函数，理解其底层原理：

```
/* strlen/strcpy/strcmp/strcat 自己实现? - 示例实现 */
size_t my_strlen(const char *s) {
    const char *p = s;
    while (*p) p++;
    return p - s;
}

char *my_strcpy(char *dst, const char *src) {
    char *ret = dst;
    while ((*dst++ = *src++) != '\0');
    return ret;
}

int my_strcmp(const char *s1, const char *s2) {
    while (*s1 && *s1 == *s2) { s1++; s2++; }
    return *(unsigned char *)s1 - *(unsigned char *)s2;
}
```

Q8: memmove vs memcpy?

💡 秒懂：memcpy直接拷贝可能在内存重叠时出错，memmove会先判断方向——就像搬家时从前往后搬和从后往前搬的区别，memmove更安全但稍慢。

两者的关键区别在于是否处理内存重叠：

```
/* memcpy: 不处理内存重叠，速度快 */
/* memmove: 处理内存重叠(可能从后往前拷贝) */
void *my_memmove(void *dst, const void *src, size_t n) {
    char *d = dst;
    const char *s = src;
    if (d < s) {
        while (n--) *d++ = *s++;          /* 从前往后 */
    } else {
        d += n; s += n;
        while (n--) *--d = *--s;          /* 从后往前 */
    }
    return dst;
}
```

Q9: 旋转数组(右移 k 位)?

 **秒懂：** 旋转数组就像队伍整体右移k位——最巧妙的方法是三次反转：先整体反转，再反转前k个，再反转剩下的， $O(n)$ 时间 $O(1)$ 空间。

把数组 `[1,2,3,4,5,6,7]` 右移 3 位变成 `[5,6,7,1,2,3,4]`。最巧妙的方法是**三次反转法**：先把整个数组反转变成 `[7,6,5,4,3,2,1]`，再把前 k 个反转变成 `[5,6,7,4,3,2,1]`，最后把剩下的反转变成 `[5,6,7,1,2,3,4]`。本质上是利用两次反转等于不变的数学性质。注意 k 要对 n 取模，防止 $k > n$ 的情况。时间 $O(n)$ ，空间 $O(1)$ ，原地完成。

```
void rotate(int *arr, int n, int k) {
    k %= n;
    reverse(arr, 0, n - 1);    /* 全部反转 */
    reverse(arr, 0, k - 1);    /* 前k个反转 */
    reverse(arr, k, n - 1);    /* 后面反转 */
}
```

Q10: 找数组中出现次数超过一半的数(摩尔投票法)?

 **秒懂：** 摩尔投票法就像'打擂台'——每个数和当前候选人PK，相同加分不同减分，减到0换人，最终剩下的就是超过一半的那个数。

这道题核心思想是**抵消**：想象不同数字之间互相"抵消"，最后剩下的一定是出现超过一半的那个数。算法维护一个候选人 `candidate` 和计数 `count`：遇到相同的数 `count++`，遇到不同的 `count--`，`count` 减到 0 就换候选人。因为多数元素超过一半，抵消完一定剩它。时间 $O(n)$ ，空间 $O(1)$ 。

```
int majorityElement(int *arr, int n) {
    int candidate = arr[0], count = 1;
    for (int i = 1; i < n; i++) {
        if (count == 0) { candidate = arr[i]; count = 1; }
        else if (arr[i] == candidate) count++;
        else count--;
    }
    return candidate; /* 题目保证一定存在 */
}
```

Q11: 合并两个有序数组?

 **秒懂：** 合并两个有序数组就像两队人按身高合并成一队——两个指针分别指向两个数组，每次取较小的放入结果，谁先走完把另一个剩余部分直接接上。

两个已排序数组 `a[m]` 和 `b[n]`，合并到 `a` 中 (`a` 有足够空间)。关键技巧是**从后往前填**：用三个指针分别指向 `a` 的末尾、`b` 的末尾和结果的末尾，每次取较大的放到结果末尾。这样可以原地合并不需要额外空间，也不会覆盖还没处理的数据。如果从前往后填，就需要移动元素，复杂度变成 $O(m*n)$ 。

```

void merge(int *a, int m, int *b, int n) {
    int i = m - 1, j = n - 1, k = m + n - 1;
    while (i >= 0 && j >= 0) {
        a[k--] = (a[i] > b[j]) ? a[i--] : b[j--];
    }
    while (j >= 0) a[k--] = b[j--]; /* b 有剩余 */
    /* a 有剩余不用处理，已经在原位 */
}

```

Q12: KMP 字符串匹配?

 **秒懂：** KMP就像聪明地找书中的关键词——匹配失败时不从头开始，而是利用已匹配部分的信息（next数组）跳过不必要的比较，从 $O(mn)$ 降到 $O(m+n)$ 。

暴力匹配在不匹配时主串指针要回退，KMP 的核心改进是：**主串指针永远不回退**，只移动模式串指针。这靠预处理出 next 数组实现——next[j] 表示模式串前 j 个字符的最长相等前后缀长度。当位置 j 失配时，模式串跳到 next[j] 位置继续匹配，相当于利用已匹配的信息跳过不可能成功的比较。时间复杂度 $O(m+n)$ ，其中 m 是主串长，n 是模式串长。

```

void build_next(const char *pat, int *next, int n) {
    next[0] = 0;
    int len = 0, i = 1;
    while (i < n) {
        if (pat[i] == pat[len]) { next[i++] = ++len; }
        else if (len > 0) { len = next[len - 1]; }
        else { next[i++] = 0; }
    }
}

int kmp_search(const char *text, const char *pat) {
    int m = strlen(text), n = strlen(pat);
    int next[n];
    build_next(pat, next, n);
    int i = 0, j = 0;
    while (i < m) {
        if (text[i] == pat[j]) { i++; j++; }
        else if (j > 0) { j = next[j - 1]; }
        else { i++; }
        if (j == n) return i - n; /* 找到 */
    }
    return -1;
}

```

Q13: atoi 实现?

 **秒懂：** atoi就像人工把'123'变成数字123——要处理的边界多到爆：前导空格、正负号、非数字字符、溢出（超过INT_MAX/INT_MIN），面试重点考细心。

把字符串 `"-123abc"` 转为整数 `-123`。看起来简单但边界极多：(1) 跳过后导空格 (2) 处理正负号 (3) 逐字符转数字 (4) 遇到非数字字符停止 (5) **溢出检测**是最关键的一一在乘10加新数字之前要判断是否会超过 `INT_MAX / INT_MIN`。面试中经常考这道题来看你对边界条件的处理能力。

```
int my_atoi(const char *s) {
    while (*s == ' ') s++;           /* 跳空格 */
    int sign = 1;
    if (*s == '-' || *s == '+') { sign = (*s == '-') ? -1 : 1; s++; }
    long result = 0;
    while (*s >= '0' && *s <= '9') {
        result = result * 10 + (*s - '0');
        if (result * sign > INT_MAX) return INT_MAX;
        if (result * sign < INT_MIN) return INT_MIN;
        s++;
    }
    return (int)(result * sign);
}
```

Q14: 大数相加(字符串模拟)?

 **秒懂**：大数相加就像小学竖式加法——从最低位对齐逐位相加，进位向前传，结果用字符串存储，不受整数位数限制。

当两个数大到 `long long` 都装不下时（如 100 位数），只能用字符串模拟手工加法：从末尾开始逐位相加，处理进位，结果倒序存储最后反转。这在嵌入式中不常考，但在算法面试中是基础题。核心就是模拟我们小学学的竖式加法。

```
char *addStrings(const char *a, const char *b) {
    int i = strlen(a) - 1, j = strlen(b) - 1;
    int carry = 0, k = 0;
    char result[1024];
    while (i >= 0 || j >= 0 || carry) {
        int sum = carry;
        if (i >= 0) sum += a[i--] - '0';
        if (j >= 0) sum += b[j--] - '0';
        result[k++] = sum % 10 + '0';
        carry = sum / 10;
    }
    result[k] = '\0';
    /* 反转 result */
    for (int l = 0, r = k - 1; l < r; l++, r--) {
        char t = result[l]; result[l] = result[r]; result[r] = t;
    }
    return strdup(result);
}
```

Q15: 最长无重复字符子串(滑动窗口)?

 **秒懂：** 滑动窗口就像一个可伸缩的框在数组上滑动——用哈希记录窗口内字符，遇到重复就收缩左边界，维护最大窗口长度。

给定字符串找最长的不含重复字符的连续子串。用**滑动窗口**：维护左右指针 `[left, right]`，`right` 不断右移扩大窗口。用一个数组(哈希表)记录每个字符是否在窗口中。当新字符已存在时，不断收缩左边界直到没有重复。每次更新最大长度。时间 $O(n)$ ，因为 `left` 和 `right` 各自最多走 n 步。

```
int lengthOfLongestSubstring(const char *s) {
    int count[128] = {0};    /* ASCII 字符计数 */
    int left = 0, max_len = 0;
    for (int right = 0; s[right]; right++) {
        count[(int)s[right]]++;
        while (count[(int)s[right]] > 1) {
            count[(int)s[left]]--;
            left++;
        }
        int len = right - left + 1;
        if (len > max_len) max_len = len;
    }
    return max_len;
}
```

Q16: 螺旋矩阵遍历?

 **秒懂：** 螺旋遍历就像剥洋葱——从外圈到内圈，每圈按右→下→左→上的顺序走，用四个边界变量控制，走完一边就收缩对应边界。

按照右→下→左→上的顺序一圈一圈地遍历二维矩阵。维护四个边界：`top/bottom/left/right`，每遍历完一条边就收缩对应边界。当上下边界交叉或左右边界交叉时停止。面试中要注意矩阵不是正方形的情况（如 3×5 ），某些方向可能提前结束。

```
void spiralOrder(int mat[][N], int rows, int cols) {
    int top = 0, bot = rows - 1, left = 0, right = cols - 1;
    while (top <= bot && left <= right) {
        for (int j = left; j <= right; j++) printf("%d ", mat[top][j]);
        top++;                                // 入栈
        for (int i = top; i <= bot; i++) printf("%d ", mat[i][right]);
        right--;
        if (top <= bot) {
            for (int j = right; j >= left; j--) printf("%d ", mat[bot][j]);
            bot--;
        }
        if (left <= right) {
            for (int i = bot; i >= top; i--) printf("%d ", mat[i][left]);
            left++;
        }
    }
}
```


Q17: 二维有序数组查找?

 **秒懂：** 二维有序数组查找（杨氏矩阵）——从右上角开始，目标大了往下走，目标小了往左走，像走楼梯一样， $O(m+n)$ 就能找到。

一个 $m \times n$ 矩阵，每行从左到右递增，每列从上到下递增，查找目标值。从**右上角**出发：如果当前值等于目标就找到了；如果当前值大于目标，说明这整列都太大，往左走；如果当前值小于目标，说明这整行都太小，往下走。每步淘汰一行或一列，时间 $O(m+n)$ 。这比暴力 $O(mn)$ 和逐行二分 $O(m \log n)$ 都快。

```
int searchMatrix(int mat[][N], int rows, int cols, int target) {
    int i = 0, j = cols - 1; /* 右上角出发 */
    while (i < rows && j >= 0) {
        if (mat[i][j] == target) return 1;
        else if (mat[i][j] > target) j--;
        else i++;
    }
    return 0;
}
```

Q18: 字符串转整数的边界处理?

 **秒懂：** 字符串转整数的边界就像走钢丝——前导空格要跳过，正负号只能有一个，遇到非数字停止，最重要的是溢出检测要在乘10之前判断。

这是 Q13 的面试追问重点。面试官关心你能否处理所有边界：空字符串、全空格、只有正负号("+")、前导零("00123")、溢出("999999999999")、中间出现非数字("12a3" → 12)。最关键的是溢出判断：在 `result = result * 10 + digit` 之前检查 `result > (INT_MAX - digit) / 10`。建议面试时先列出所有边界case再写代码。

```
/* 字符串转整数(atoi) - 处理边界情况 */
int myAtoi(const char *s) {
    while (*s == ' ') s++;           // 1. 跳过前导空格
    int sign = 1;
    if (*s == '-' || *s == '+')      // 2. 处理正负号
        sign = (*s++ == '-') ? -1 : 1;
    long result = 0;
    while (*s >= '0' && *s <= '9') { // 3. 逐位转换
        result = result * 10 + (*s++ - '0');
        if (result * sign > INT_MAX) return INT_MAX; // 4. 溢出检查
        if (result * sign < INT_MIN) return INT_MIN;
    }
    return (int)(result * sign);
}

/* 关键边界：溢出/空串/非数字字符/正负号/前导空格 */
```

Q19: 数组中第 K 大元素(快速选择)?

 **秒懂：** 快速选择就像考试只需找班级第K名不需要全排——基于快排partition，每次只递归一半，平均 $O(n)$ 就能找到第K大，比排序的 $O(n\log n)$ 快。

排序后取第 K 大需要 $O(n\log n)$ ，但**快速选择**算法平均 $O(n)$ ：类似快排的 partition，每次只递归一半。partition 把数组分成"比 pivot 大"和"比 pivot 小"两部分，如果 pivot 刚好在第 K 个位置就找到了；如果 K 在左半边就只处理左半边，右半边直接丢弃。平均每次问题规模减半： $n + n/2 + n/4 + \dots = O(n)$ 。

```
int partition(int *arr, int lo, int hi) {
    int pivot = arr[hi], i = lo;                // 基准元素(快排)
    for (int j = lo; j < hi; j++) {
        if (arr[j] >= pivot) { /* 降序partition, 大的在前 */
            int t = arr[i]; arr[i] = arr[j]; arr[j] = t;
            i++;
        }
    }
    int t = arr[i]; arr[i] = arr[hi]; arr[hi] = t;
    return i;
}

int findKthLargest(int *arr, int n, int k) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int p = partition(arr, lo, hi);
        if (p == k - 1) return arr[p];
        else if (p < k - 1) lo = p + 1;
        else hi = p - 1;
    }
    return -1;
}
```

Q20: 子数组最大和(Kadane 算法)?

 **秒懂：** Kadane算法就像走路记账——每走一步决定是'接着前面的路走'还是'重新开始'，取两者较大值，同时更新全局最大值， $O(n)$ 搞定。

给定数组 `[-2,1,-3,4,-1,2,1,-5,4]`，找连续子数组的最大和（答案是 `[4,-1,2,1]=6`）。Kadane 算法的思想很简单：维护 `current_sum` 表示以当前元素结尾的最大子数组和。对每个元素，要么加入前面的子数组(`current_sum + arr[i]`)，要么自己单独开始一个新子数组(`arr[i]`)，取较大值。同时更新全局最大值。只需遍历一次，时间 $O(n)$ 空间 $O(1)$ 。

```
int maxSubArray(int *arr, int n) {
    int current_sum = arr[0], max_sum = arr[0];
    for (int i = 1; i < n; i++) {
        /* 要么接上前面，要么自己重新开始 */
        current_sum = (current_sum + arr[i] > arr[i]) ?
            current_sum + arr[i] : arr[i];
        if (current_sum > max_sum) max_sum = current_sum;
    }
    return max_sum;
}
```

二、链表（Q21~Q48）

Q21: 单链表定义？

 **秒懂：** 单链表就像火车车厢——每节车厢（节点）装数据和指向下节车厢的钩子（指针），只能从头往后走不能倒退，插入删除只需改钩子O(1)。

链表操作的核心是指针的修改：

```
typedef struct ListNode {
    int val;
    struct ListNode *next;
} ListNode;

/* 创建节点 */
ListNode *new_node(int val) {
    ListNode *node = (ListNode *)malloc(sizeof(ListNode)); // 动态分配节点
    node->val = val;
    node->next = NULL; // 修改指针指向
    return node;
}

/* 头插法建链表 */
ListNode *head = NULL;
for (int i = 0; i < n; i++) {
    ListNode *node = new_node(data[i]);
    node->next = head; // 修改指针指向
    head = node;
}
```

Q22: 反转链表(最高频！)

 **秒懂：** 反转链表是面试第一高频题——迭代法用三个指针：prev、curr、next，每次把当前节点指向前一个，就像逐个掉头，面试必须秒写。

使用双指针或递归方式实现：

```
ListNode *reverse(ListNode *head) {
    ListNode *prev = NULL, *curr = head;
```

```
while (curr) {
    ListNode *next = curr->next;
    curr->next = prev;           // 修改指针指向
    prev = curr;
    curr = next;
}
return prev;
}

/* 递归版 */
ListNode *reverse_rec(ListNode *head) {
    if (!head || !head->next) return head;           // 头节点的下一个
    ListNode *new_head = reverse_rec(head->next);    // 头节点的下一个
    head->next->next = head;                           // 修改指针指向
    head->next = NULL;                                // 修改指针指向
    return new_head;
}
/* 时间 O(n), 空间 O(1) / O(n) */
```

💡 **面试追问：** 递归和迭代两种写法？ 时间空间复杂度？ 在面试中应该写哪种？

🔧 **嵌入式建议：** 嵌入式面试最高频链表题!迭代版O(1)空间(推荐)。递归版简洁但栈溢出风险(嵌入式栈小)。

📊 链表 vs 数组 对比表

特性	数组	链表
内存分配	连续(编译期/一次性)	分散(运行时按节点)
随机访问	O(1)	O(n)
头部插入/删除	O(n)(需移动)	O(1)
尾部插入	O(1)(预留空间时)	O(n)/O(1)(有尾指针)
中间插入/删除	O(n)	O(1)(已知位置时)
缓存友好性	🌟 好(连续内存)	差(跳转多)
嵌入式推荐	🌟 静态数组(无碎片)	静态链表(预分配节点池)

Q23: 判断链表有环？

💡 **秒懂：** 判断链表有环用快慢指针——快指针每次走两步慢指针走一步，如果有环一定会相遇（就像操场跑步快的人会追上慢的人）。

链表操作的核心是指针的修改：

```
/* 快慢指针(Floyd 判环法) */
int hasCycle(ListNode *head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
```


Q25: 删除链表倒数第 N 个节点?

 **秒懂：** 删除倒数第N个就像'间隔N步'走路——快指针先走N步，然后快慢一起走，快到终点时慢指针恰好在倒数第N+1个位置，改next即可。

链表操作的核心是指针的修改：

```
ListNode *removeNthFromEnd(ListNode *head, int n) {
    ListNode dummy = {0, head};
    ListNode *fast = &dummy, *slow = &dummy;
    /* fast 先走 n+1 步 */
    for (int i = 0; i <= n; i++) fast = fast->next;
    /* 同步走到 fast == NULL */
    while (fast) {
        fast = fast->next;
        slow = slow->next;
    }
    ListNode *del = slow->next;
    slow->next = del->next;           // 修改指针指向
    free(del);
    return dummy.next;
}
```

Q26: 链表中间节点?

 **秒懂：** 找中间节点用快慢指针——快走两步慢走一步，快到末尾时慢正好在中间，常用于链表归并排序的分割步骤。

链表操作的核心是指针的修改：

```
ListNode *middleNode(ListNode *head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow; /* 偶数个节点时返回第二个中间 */
}
```

Q27: 判断两个链表是否相交?

 **秒懂：** 两链表相交就像两条路汇成一条——先算长度差让长的先走差值步，然后一起走，相遇的节点就是交点。或者各走完自己的走对方的，也能相遇。

链表操作的核心是指针的修改：

```

ListNode *getIntersection(ListNode *headA, ListNode *headB) {
    ListNode *pA = headA, *pB = headB;
    while (pA != pB) {
        pA = pA ? pA->next : headB;
        pB = pB ? pB->next : headA;
    }
    return pA; /* NULL = 不相交, 非 NULL = 交点 */
}
/* 原理：两指针走完自己的链表后走对方的，走过的总距离相等 */

```

Q28: 两数相加(链表表示)?

 **秒懂：** 链表两数相加就像竖式加法——两个链表从低位到高位逐位相加，用carry记录进位，注意最后还有进位要多加一个节点。

两个非负整数用链表**逆序**存储（如 342 → 2→4→3），求和返回链表。思路和大数相加一样：从头开始逐位相加加进位。因为链表已经是低位在前，所以天然对齐，不需要反转。注意最后 carry 可能为 1，需要多创建一个节点。时间 $O(\max(m,n))$ 。

```

ListNode *addTwoNumbers(ListNode *l1, ListNode *l2) {
    ListNode dummy = {0, NULL};
    ListNode *tail = &dummy;
    int carry = 0;
    while (l1 || l2 || carry) {
        int sum = carry;
        if (l1) { sum += l1->val; l1 = l1->next; }
        if (l2) { sum += l2->val; l2 = l2->next; }
        tail->next = new_node(sum % 10); // 修改指针指向
        tail = tail->next;
        carry = sum / 10;
    }
    return dummy.next;
}

```

Q29: 回文链表判断?

 **秒懂：** 回文链表判断——快慢指针找中点，反转后半部分，然后从头和从中间逐个比较，相同就是回文，检查完再把链表恢复原样。

判断链表 1→2→2→1 是否回文。不能像数组一样双向遍历，所以分三步：(1) 快慢指针找中点 (2) 反转后半段链表 (3) 前半段和反转后的后半段逐一比较。时间 $O(n)$ ，空间 $O(1)$ 。也可以用栈（空间 $O(n)$ ）但面试官更想看 $O(1)$ 做法。注意比较完后最好把链表恢复原样（再反转一次）。

```

int isPalindrome(ListNode *head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    /* slow 指向后半段起点 */
    ListNode *rev = reverse(slow); /* 反转后半段 */
    ListNode *p = head, *q = rev;
    int result = 1;

```

```

while (q) {
    if (p->val != q->val) { result = 0; break; }
    p = p->next; q = q->next;
}
reverse(rev); /* 恢复原链表 */
return result;
}

```

Q30: 链表排序(归并排序)?

 **秒懂**：链表归并排序——快慢指针分成两半，递归排序各半，再合并两个有序链表，是链表唯一能做到 $O(n\log n)$ 且 $O(1)$ 空间的排序方法。

链表不支持随机访问，所以快排效率不高（partition需要随机访问）。**归并排序**非常适合链表：用快慢指针找中点把链表分成两半，递归排序后合并两个有序链表。合并操作在链表上是 $O(1)$ 空间的（不需要额外数组），总时间 $O(n\log n)$ 。这也是面试中最常考的链表排序方法。

```

ListNode *sortList(ListNode *head) {
    if (!head || !head->next) return head;          // 头节点的下一个
    /* 快慢指针找中点(慢指针前一个位置) */
    ListNode *slow = head, *fast = head->next;      // 头节点的下一个
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    ListNode *mid = slow->next;
    slow->next = NULL; /* 断开 */
    ListNode *left = sortList(head);
    ListNode *right = sortList(mid);
    return merge(left, right); /* 合并两个有序链表(Q24) */
}

```

Q31: K 个一组翻转链表?

 **秒懂**：K个一组翻转就像把火车每K节车厢整体掉头——先数K个，翻转这一段，连接上一段和下一段，不够K个的保持原样。

给链表 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ ， $K=3$ ，结果 $3 \rightarrow 2 \rightarrow 1 \rightarrow 4 \rightarrow 5$ （最后不足K个不翻转）。思路：每次数K个节点出来翻转，翻转后接到前面的尾部，然后处理下一组。难点在于指针管理：需要记住每组的前驱节点和后继节点。这是 LeetCode hard 题，面试中能写出来说明链表操作非常熟练。

```

ListNode *reverseKGroup(ListNode *head, int k) {
    ListNode dummy = {0, head};
    ListNode *prev_group = &dummy;
    while (1) {
        ListNode *kth = prev_group;
        for (int i = 0; i < k; i++) {
            kth = kth->next;
            if (!kth) return dummy.next; /* 不足k个 */
        }
        ListNode *next_group = kth->next;
        ListNode *curr = prev_group->next, *prev = next_group;
        for (int i = 0; i < k; i++) {

```



```

        ListNode *tmp = curr->next;
        curr->next = prev;                                // 修改指针指向
        prev = curr;
        curr = tmp;
    }
    ListNode *tmp = prev_group->next; /* 原来的第一个=翻转后的最后一个 */
    prev_group->next = prev;          /* 接上翻转后的头 */
    prev_group = tmp;
}
}

```

Q32: LRU 缓存(双向链表+哈希表)?

💡秒懂：LRU缓存就像酒店前台——最近用过的放前面，最久没用的在后面，满了就淘汰最后一个。双向链表+哈希表实现O(1)的读写和淘汰。

LRU(Least Recently Used) 要求 `get` 和 `put` 都是 $O(1)$ 。用**双向链表**维护访问顺序（最近使用的放头部，最久未使用的在尾部），用**哈希表**存 `key`→链表节点的映射实现 $O(1)$ 查找。`get` 时把节点移到头部；`put` 时如果已存在就更新并移到头部，不存在就新建节点插入头部，满了就删尾部。这是面试超高频题。

```
/* 核心操作： */
/* get(key): hash找到节点 → 移到头部 → 返回值 */
/* put(key,val): hash找到 → 更新+移到头部；没找到 → 新建插头部，满了删尾 */
/* 双向链表保证 移动/删除/插入 都是  $O(1)$  */
```

Q33: 复制带随机指针的链表?

💡 秒懂：复制带随机指针的链表——先在每个节点后面插入一个拷贝节点，然后复制random指针指向，最后拆分成两条链表，巧妙避免哈希表。

每个节点除了 `next` 还有一个 `random` 指向链表中任意节点。**方法一**：哈希表存 原节点→新节点 的映射，两次遍历（第一次建节点，第二次连 `next`和`random`）。**方法二(巧妙)**：在每个原节点后面插入它的副本 `A→A'→B→B'→C→C'`，然后 `A'.random = A.random.next`（原节点的`random`的下一个就是副本），最后拆分成两个链表。方法二空间 $O(1)$ 。

```

/* 复制带随机指针的链表(LeetCode 138) */
typedef struct Node { int val; struct Node *next, *random; } Node;

/* 方法：交错插入法 O(n)时间 O(1)空间 */
Node* copyRandomList(Node* head) {
    if (!head) return NULL;
    // 1. 在每个节点后插入副本：A→A'→B→B'→C→C'
    for (Node *cur = head; cur; cur = cur->next->next) {
        Node *copy = (Node*)malloc(sizeof(Node));
        copy->val = cur->val;
        copy->next = cur->next;
        cur->next = copy;
    }
}

```

```

// 2. 设置random: A'.random = A.random.next
for (Node *cur = head; cur; cur = cur->next->next)
    cur->next->random = cur->random ? cur->random->next : NULL;
// 3. 拆分两个链表
Node *new_head = head->next; // 头节点的下一个
for (Node *cur = head; cur; cur = cur->next) {
    Node *copy = cur->next;
    cur->next = copy->next; // 修改指针指向
    copy->next = copy->next ? copy->next->next : NULL; // 修改指针指向
}
return new_head;
}

```

Q34: 双向链表插入/删除?

 **秒懂：** 双向链表每个节点有prev和next两个指针——像双向车道，前后都能走，插入删除更方便（知道节点就能O(1)操作），Linux内核大量使用。

双向链表每个节点有 `prev` 和 `next` 两个指针。**优势：** 删除操作不需要遍历找前驱（单链表删除需要知道前一个节点）。插入需要修改4个指针，删除也是4个指针。双向链表是 LRU 缓存和 Linux 内核链表的基础数据结构。

```

/* 在 node 之后插入 new_node */
void insert_after(ListNode2 *node, ListNode2 *new_node) {
    new_node->next = node->next; // 修改指针指向
    new_node->prev = node;
    if (node->next) node->next->prev = new_node;
    node->next = new_node; // 修改指针指向
}
/* 删除 node(已知位置) */
void delete_node(ListNode2 *node) {
    if (node->prev) node->prev->next = node->next; // 修改指针指向
    if (node->next) node->next->prev = node->prev;
    free(node);
}

```

Q35: 循环链表(约瑟夫环)?

 **秒懂：** 约瑟夫环就像'丢手帕'游戏——n个人围成圈每次数m个淘汰一个，用循环链表模拟，或者用递推公式 $f(n,m)=(f(n-1,m)+m)\%n$ 直接算。

n 个人围成圈，从第 1 个人开始报数，报到 m 的人出列，然后从下一个人继续。求最后剩下谁。用循环链表模拟：建立 n 个节点的环形链表，每次走 m-1 步删除当前节点。也可以用数学公式递推： $f(1)=0$ ， $f(n)=(f(n-1)+m)\%n$ ，结果+1就是编号。

```
int josephus(int n, int m) {
    int result = 0; /* f(1) = 0 */
    for (int i = 2; i <= n; i++) {
        result = (result + m) % i;
    }
    return result + 1; /* 转为1-based编号 */
}
```

Q36: 跳表(Skip List)?

 **秒懂：** 跳表就像给链表加了'快车道'——多层索引，查找时从最高层开始往右走，走不动就下一层，平均 $O(\log n)$ 查找，Redis 的有序集合就用跳表。

跳表是对**有序链表**的加速结构。普通有序链表查找 $O(n)$ 太慢。跳表建立多层"快车道"——底层包含所有元素，每上一层约保留 1/2 的元素(随机决定)。查找时从顶层开始，先向右走到不能走(下一个比目标大)，再往下走一层继续向右，逐层缩小范围。

```
Level 3: 1 —————> 9 —> nil
Level 2: 1 —> 4 —————> 9 —> nil
Level 1: 1 -> 3 -> 4 -> 5 -> 7 -> 9 -> nil
```

查找平均 $O(\log n)$ ，插入/删除也是 $O(\log n)$ 。Redis 的有序集合(ZSET)底层就用跳表——比平衡树实现简单、范围查询方便。

Q37: 有序链表去重?

 **秒懂：** 有序链表去重就像排队时踢掉重复的人——当前和下一个相同就删下一个，一次遍历 $O(n)$ 搞定，注意要释放删除节点的内存。

遍历链表，如果当前节点的值和下一个节点相同，就删除下一个节点：

```
void remove_duplicates(Node *head) {
    Node *curr = head;
    while (curr && curr->next) {
        if (curr->val == curr->next->val) {
            Node *dup = curr->next;
            curr->next = dup->next;           // 修改指针指向
            free(dup);
        } else {
            curr = curr->next;
        }
    }
}
```

/* [1,1,2,3,3] -> [1,2,3] */

时间 $O(n)$ ，空间 $O(1)$ 。因为链表有序，相同元素必然相邻。

Q38: 无序链表去重?

 **秒懂：** 无序链表去重需要用哈希表记录已出现的值——遍历链表，没见过的加入哈希表，见过的删除， $O(n)$ 时间 $O(n)$ 空间。

无序链表中相同元素可能不相邻，需要用**哈希表(或数组)**记录已出现的值：

```
void remove_dup_unsorted(Node *head) {
    int seen[10000] = {0}; /* 简单用数组做哈希 */
    Node *prev = NULL, *curr = head;
    while (curr) {
        if (seen[curr->val]) {
            prev->next = curr->next;          // 修改指针指向
            free(curr);
            curr = prev->next;
        } else {
            seen[curr->val] = 1;
            prev = curr;
            curr = curr->next;
        }
    }
}
```

时间 $O(n)$ ，空间 $O(n)$ 。不用额外空间的话只能双重循环 $O(n^2)$ 。

Q39: 分隔链表(按值分区)?

 **秒懂：** 分隔链表就像根据身高分成两队——小于 x 的站左边，大于等于 x 的站右边，最后接起来，用两个虚拟头节点分别收集再连接。

给定值 x ，把链表分成"所有小于 x 的节点在前、大于等于 x 的在后"(保持原序)。方法：创建两个虚拟链表 `less` 和 `greater`，遍历原链表把每个节点挂到对应链表上，最后拼接：

```
Node *partition(Node *head, int x) {
    Node less_head = {0}, greater_head = {0};
    Node *less = &less_head, *greater = &greater_head;
    while (head) {
        if (head->val < x) { less->next = head; less = less->next; } // 修改指针指向
        else { greater->next = head; greater = greater->next; } // 修改指针指向
        head = head->next; // 头节点的下一个
    }
    greater->next = NULL; // 修改指针指向
    less->next = greater_head.next; // 修改指针指向
    return less_head.next;
}
```

Q40: 旋转链表(右移k位)

 **秒懂：** 旋转链表就像把项链断开重接——先连成环，然后在正确位置断开，关键是k要对长度取模，避免转了整圈白忙活。

先遍历得到长度 n 和尾节点， $k \% n$ ，把尾节点接到头部形成环，然后在第 $n-k$ 个节点处断开。例如 $[1,2,3,4,5]$ 右移 $2 \rightarrow [4,5,1,2,3]$ 。

平衡二叉树(AVL 树)：每个节点左右子树高度差不超过 1。插入/删除后如果失衡,通过旋转(左旋/右旋/左右旋/右左旋)恢复平衡。查找/插入/删除都是 $O(\log n)$ 。比普通 BST 保证了最坏情况性能。缺点：频繁插入删除时旋转操作多。红黑树放宽了平衡条件(最长路径不超过最短的 2 倍),旋转更少。

```
/* AVL 右旋 */
Node* rightRotate(Node* y) {
    Node* x = y->left;
    y->left = x->right;           // 指向左子树
    x->right = y;                // 指向右子树
    y->height = 1 + max(height(y->left), height(y->right));
    x->height = 1 + max(height(x->left), height(x->right));
    return x;
}
```

Q41: 删除所有等于 val 的节点?

 **秒懂：** 删除所有等于val的节点要用虚拟头节点——因为头节点也可能要删，用prev和curr双指针遍历，curr等于val就跳过。

用哨兵头节点(dummy)，遍历时检查 `curr->next->val`，命中则跳过并 free。哨兵头节点简化了"删除头节点"的特殊情况处理。

红黑树规则：(1) 节点是红或黑 (2) 根是黑 (3) 叶子(NIL)是黑 (4) 红节点的子节点是黑(不能连续红) (5) 从任一节点到叶子的路径包含相同数量的黑节点。插入新节点默认红色,通过变色和旋转修复违规。Linux 内核大量使用(CFS 调度器/内存管理/epoll)。

红黑树节点颜色规则：

```
      B(10)
     /  \
    R(5)  R(15)
   / \   / \
  B(3) B(7) B(12) B(20)
```

规则：红节点的子节点必须是黑色

Q42~Q43: 链表表示的两数相加： 逆序(LeetCode 2)直接逐位相加进位；正序需要先反转两个链表再相加再反转回来(或用栈辅助)。

Q42: 扁平化多级双向链表?

 **秒懂：** 扁平化多级双向链表就像把嵌套文件夹展开成一维列表——遇到child就递归或用栈展开，把child链表插入next位置。

节点有 next 和 child 两个指针。DFS 处理——遇到 child 就递归展开并插入当前位置之后，原 next 接到展开后的尾部。

B 树：每个节点可以有多个键和子节点(阶为 m 则每个节点最多 $m-1$ 个键和 m 个子节点)。所有叶子在同一层。查找从根开始,在节点内二分查找确定进入哪个子节点。适合磁盘存储——每个节点对应一个磁盘块,减少 IO 次数。B+ 树的数据只在叶子节点,叶子节点用链表连接——范围查询高效。

Q43: 链表快速排序?

 **秒懂：** 链表快速排序——选头节点做pivot，遍历一次分成小于和大于两部分，递归排序后连接，比数组快排少了swap但多了分组操作。

选头节点做 pivot，遍历分成 $< pivot / = pivot / > pivot$ 三个子链表，递归排序后拼接。不过链表排序更推荐归并排序(Q118已介绍)。

字典树(Trie)：每个节点代表一个字符,从根到某节点的路径组成一个前缀。用于字符串前缀匹配(自动补全/拼写检查/IP路由最长匹配)。空间消耗大(每个节点 26 个子指针)——用压缩 Trie(Radix Tree)合并只有一个子节点的路径。Linux 内核页表管理用 Radix Tree。

Q44: 两两交换相邻节点?

 **秒懂：** 两两交换就像舞伴互换位置——每次取两个节点交换顺序，用prev指针处理连接关系，注意处理好前后指针更新。

$[1,2,3,4] \rightarrow [2,1,4,3]$ 。用哨兵头节点，每次操作两个节点： $prev \rightarrow A \rightarrow B \rightarrow C$ 变成 $prev \rightarrow B \rightarrow A \rightarrow C$ ，然后 prev 跳到 A 继续。

图的存储：(1) 邻接矩阵——二维数组 $G[i][j]$ =权重,查询边 $O(1)$,空间 $O(V^2)$ (稀疏图浪费) (2) 邻接表——每个顶点一个链表(或数组)存储相邻顶点,空间 $O(V+E)$,遍历邻居高效。稠密图用矩阵,稀疏图用邻接表。嵌入式中图不常用,但路由/状态机可抽象为图。

Q45: 反转链表的 m 到 n 区间?

 **秒懂：** 反转m到n区间——先走到m位置，然后用头插法反转m到n这段，最后把断开的三段重新接上，一次遍历 $O(n)$ 。

先走到第 $m-1$ 个节点(反转段的前驱)，对 $m \sim n$ 段做反转(Q22的方法)，再把反转后的段接回原链表。

DFS(深度优先搜索)：用栈(递归)实现,沿一条路径走到底再回溯。用于：拓扑排序/连通分量/路径查找。BFS(广度优先搜索)：用队列实现,逐层扩展。用于：最短路径(无权图)/层序遍历。时间复杂度都是 $O(V+E)$ 。DFS 空间 $O(V)$ (递归深度),BFS 空间 $O(V)$ (队列大小)。

```
/* BFS 模板 */
queue<int> q;
q.push(start); visited[start] = true;           // 访问标记数组
while (!q.empty()) {
    int u = q.front(); q.pop();                 // 队头指针
    for (int v : adj[u])
        if (!visited[v]) { visited[v] = true; q.push(v); } // 访问标记数组
}
```

Q46: 奇偶链表分离？

 **秒懂：** 奇偶链表分离——把奇数位和偶数位的节点分别串成两条链表，最后奇链尾接偶链头，一次遍历 $O(n)$ 空间 $O(1)$ 。

把下标为奇数的节点放前面、偶数的放后面。维护 odd 和 even 两个指针各自串联奇偶节点，最后 odd 尾接 even 头。

Dijkstra 算法求单源最短路径(无负权边)。贪心策略：每次选距离最小的未访问节点,更新其邻居的最短距离。用优先队列(最小堆)优化后时间 $O((V+E)\log V)$ 。不能处理负权边——用 Bellman-Ford($O(VE)$)。嵌入式中用于网络路由(OSPF 协议)。

Q47: 重排链表？

 **秒懂：** 重排链表 $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$ ——先找中点切成两半，反转后半部分，然后交替合并前后两半，三步走经典操作。

$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$ 三步：(1) 找中点(快慢指针) (2) 反转后半部分 (3) 交叉合并前后两半。

最小生成树(MST)：连通无向图中权值和最小的树。Kruskal 算法：边排序后贪心选最小边(不成环——用并查集判断), $O(E \log E)$ 。Prim 算法：从一个顶点开始,每次选连接已选和未选顶点的最小边,用最小堆 $O(E \log V)$ 。稀疏图 Kruskal 好,稠密图 Prim 好。

Q48: 链表在嵌入式中的应用？

 **秒懂：** 链表在嵌入式中很常见——RTOS任务链表、内存池空闲块链表、消息队列，嵌入式用静态数组模拟链表可以避免动态内存分配的碎片问题。

Linux 内核用 `list_head` 侵入式双向循环链表——链表节点嵌入在数据结构体中(而不是数据指针存在链表节点里)，用 `container_of` 宏从节点反推数据。FreeRTOS 就绪列表、延迟列表也是链表实现。

拓扑排序：对 DAG(有向无环图)排列节点使所有边从前指向后。Kahn 算法：入度为 0 的入队,每次取一个移除其出边(减邻居入度),入度变 0 的入队。如果排出的节点数 < 总节点数说明有环。用于：任务依赖调度/编译顺序/Makefile 依赖分析。时间 $O(V+E)$ 。

三、栈与队列（Q49~Q64）

Q49: 栈的特性和实现？

 **秒懂：** 栈就像一摞盘子——只能从顶部放入 (push) 和取出 (pop)，后进先出 (LIFO)，用数组或链表都能实现，函数调用就靠栈来管理。

```
/* 数组实现 */
#define MAX_SIZE 100
typedef struct { // 结构体定义
    int data[MAX_SIZE];
    int top;
} Stack;

void stack_init(Stack *s) { s->top = -1; }
int stack_empty(Stack *s) { return s->top == -1; }
int stack_full(Stack *s) { return s->top == MAX_SIZE - 1; }
```

```
void stack_push(Stack *s, int val) {
    if (!stack_full(s))
        s->data[++s->top] = val;
}
int stack_pop(Stack *s) {
    if (!stack_empty(s))
        return s->data[s->top--]; // 出栈
    return -1;
}
int stack_peek(Stack *s) {
    if (!stack_empty(s))
        return s->data[s->top];
    return -1;
}
```

十大排序算法复杂度速查表：

排序算法	平均O	最好O	最坏O	空间O	稳定性	嵌入式适用
冒泡排序	n^2	n	n^2	1	稳定	小数据量教学
选择排序	n^2	n^2	n^2	1	不稳定	交换次数少
插入排序	n^2	n	n^2	1	稳定	近似有序/小数据
希尔排序	n^1.3	n	n^2	1	不稳定	中等数据量
归并排序	nlogn	nlogn	nlogn	n	稳定	外部排序
快速排序	nlogn	nlogn	n^2	logn	不稳定	通用首选
堆排序	nlogn	nlogn	nlogn	1	不稳定	内存受限
计数排序	n+k	n+k	n+k	k	稳定	整数小范围
桶排序	n+k	n+k	n^2	n+k	稳定	均匀分布浮点
基数排序	n*k	n*k	n*k	n+k	稳定	定长整数/字符串

嵌入式面试重点：堆排序O(1)空间+O(nlogn)时间，最适合内存受限的MCU场景。快排在平均情况最快但最坏O(n^2)且不稳定。如果需要稳定排序且内存充足选归并。

Q50: 队列的实现(环形数组)?

秒懂： 环形队列就像传送带——用数组+头尾指针实现，尾追上头就满了，空间循环利用不浪费，嵌入式里UART缓冲区、RTOS消息队列都用这个。

基本数据结构的实现：

```
/* 队列的实现(环形数组)? - 示例实现 */
#define QSIZE 100
typedef struct { // 结构体定义
    int data[QSIZE];
```



```

    int head, tail, count;
} Queue;

void queue_init(Queue *q) { q->head = q->tail = q->count = 0; }
int queue_empty(Queue *q) { return q->count == 0; }
int queue_full(Queue *q) { return q->count == QSIZE; }

void enqueue(Queue *q, int val) {
    if (!queue_full(q)) {
        q->data[q->tail] = val;
        q->tail = (q->tail + 1) % QSIZE;
        q->count++;
    }
}

int dequeue(Queue *q) {
    if (!queue_empty(q)) {
        int val = q->data[q->head];
        q->head = (q->head + 1) % QSIZE;
        q->count--;
        return val;
    }
    return -1;
}

```

Q51: 有效括号匹配?

 **秒懂**：括号匹配用栈天然适合——遇到左括号压栈，遇到右括号弹栈检查是否配对，最后栈为空则全部匹配，是栈的经典应用。

括号匹配是栈的经典应用，面试手撕代码高频题：

```

/* 有效括号匹配? - 示例实现 */
int isValid(const char *s) {
    Stack st;
    stack_init(&st);
    while (*s) {
        if (*s == '(' || *s == '[' || *s == '{') {
            stack_push(&st, *s);
        } else {
            if (stack_empty(&st)) return 0;
            char top = stack_pop(&st);
            if ((*s == ')' && top != '(') ||
                (*s == ']' && top != '[') ||
                (*s == '}' && top != '{'))
                return 0;
        }
        s++;
    }
    return stack_empty(&st);
}

```

★ 常用数据结构选型（嵌入式场景）：

数据结构	查找	插入/删除	内存	嵌入式应用
数组	O(1)随机/O(n)查找	O(n)	连续,紧凑	查找表/配置参数
链表	O(n)	O(1)	离散,有指针开销	动态队列/内存池
栈	O(1)栈顶	O(1)	小	函数调用/表达式解析
队列	O(1)队头	O(1)	小	消息队列/环形缓冲区
哈希表	O(1)平均	O(1)平均	有浪费	缓存/符号表
二叉搜索树	O(logn)	O(logn)	指针开销	有序数据管理
堆	O(1)最值	O(logn)	数组实现紧凑	优先级队列/定时器管理
红黑树	O(logn)	O(logn)	平衡保证	Linux内核(调度/内存)

★ 排序算法对比（面试必背）：

算法	平均	最坏	最好	空间	稳定	嵌入式适用
冒泡	O(n²)	O(n²)	O(n)	O(1)	✓	超小数据
选择	O(n²)	O(n²)	O(n²)	O(1)	✗	数据交换少时
插入	O(n²)	O(n²)	O(n)	O(1)	✓	近有序小数据★
希尔	O(n¹.³)	O(n²)	O(n)	O(1)	✗	中等数据
快排	O(nlogn)	O(n²)	O(nlogn)	O(logn)	✗	通用首选★
归并	O(nlogn)	O(nlogn)	O(nlogn)	O(n)	✓	外排序/链表排序
堆排	O(nlogn)	O(nlogn)	O(nlogn)	O(1)	✗	Top-K问题★
计数	O(n+k)	O(n+k)	O(n+k)	O(k)	✓	范围已知整数

嵌入式建议： MCU上内存有限,优先用插入排序(数据量小)或快排(数据量大)。堆排序适合寻找前N个最大/最小值(只维护大小为N的堆)。归并排序需要O(n)额外空间,MCU上慎用。

Q52: 用两个栈实现队列？

💡 秒懂： 两个栈实现队列——栈A负责入队（push到A），栈B负责出队（B空时把A全部倒入B再pop），就像两个杯子倒水实现先进先出。

经典面试题(栈后进先出→队列先进先出)：

```
typedef struct {
    stack in_stack;    // 入队用
    stack out_stack;   // 出队用
} MyQueue;

void queue_push(MyQueue *q, int val) {
```

```

    stack_push(&q->in_stack, val);
}

int queue_pop(MyQueue *q) {
    if (stack_empty(&q->out_stack)) {
        // out_stack空时,把in_stack全部倒过来
        while (!stack_empty(&q->in_stack)) {
            stack_push(&q->out_stack, stack_pop(&q->in_stack));
        }
    }
    return stack_pop(&q->out_stack);
}
// 均摊时间复杂度: O(1)

```

Q53: 单调栈的应用?

 **秒懂:** 单调栈就像排队看身高——维护一个递增或递减的栈，能快速找到每个元素左/右第一个更大/更小的值，O(n)解决'下一个更大元素'问题。

单调栈解决"找下一个更大/更小元素"问题:

```

// 题目: 每日温度,找到每个位置后面第一个更高温度的天数
void dailyTemperatures(int *T, int n, int *result) {
    int stack[n], top = -1;
    for (int i = 0; i < n; i++) {
        while (top >= 0 && T[i] > T[stack[top]]) {
            int idx = stack[top--];
            result[idx] = i - idx;
        }
        stack[++top] = i; // 存下标
    }
}
// 时间O(n),每个元素最多入栈出栈各一次

```

Q54: 最小栈(O(1)获取最小值)?

 **秒懂:** 最小栈用一个辅助栈同步记录当前最小值——每次push时把当前最小值也压入辅助栈，pop时同步弹出，getMin就是辅助栈顶，O(1)。

```

typedef struct { // 结构体定义
    int data[MAX_SIZE];
    int min_stack[MAX_SIZE]; // 辅助栈记录最小值
    int top;
    int min_top;
} MinStack;

void push(MinStack *s, int val) {
    s->data[++s->top] = val;
    if (s->min_top < 0 || val <= s->min_stack[s->min_top])
        s->min_stack[++s->min_top] = val;
}

```

```

void pop(MinStack *s) {
    if (s->data[s->top] == s->min_stack[s->min_top])
        s->min_top--;
    s->top--;
}

int getMin(MinStack *s) { return s->min_stack[s->min_top]; }

```

Q55: 优先级队列(堆实现)?

 **秒懂：** 优先级队列用堆实现——就像急诊室按严重程度排序而不是先来后到，插入和取最值都是 $O(\log n)$ ，RTOS调度器选最高优先级任务就靠这个。

```

typedef struct { // 结构体定义
    int data[MAX_SIZE];
    int size;
} MinHeap;

void heap_push(MinHeap *h, int val) {
    h->data[h->size] = val;
    // 上浮
    int i = h->size++;
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (h->data[i] < h->data[parent]) {
            int tmp = h->data[i]; h->data[i] = h->data[parent]; h->data[parent] = tmp;
            i = parent;
        } else break;
    }
}

int heap_pop(MinHeap *h) {
    int val = h->data[0];
    h->data[0] = h->data[--h->size];
    // 下沉
    int i = 0;
    while (2*i+1 < h->size) {
        int child = 2*i+1;
        if (child+1 < h->size && h->data[child+1] < h->data[child]) child++;
        if (h->data[i] > h->data[child]) {
            int tmp = h->data[i]; h->data[i] = h->data[child]; h->data[child] = tmp;
            i = child;
        } else break;
    }
    return val;
}

```

Q56: 栈在表达式求值中的应用?

 **秒懂**: 表达式求值用两个栈——一个放数字一个放运算符, 遇到运算符比较优先级决定是压栈还是先算, 遇到右括号把到左括号之间的都算完。

中缀表达式求值(两个栈: 数栈+运算符栈):

```
// 简化版: 只支持 + - * / ()
int calculate(const char *s) {
    int num_stack[100], op_stack[100];
    int nt = -1, ot = -1;

    auto int priority(char c) { return (c == '+' || c == '-') ? 1 : 2; }
    auto void compute() {
        int b = num_stack[nt--], a = num_stack[nt--];
        char op = op_stack[ot--];
        switch(op) {
            case '+': num_stack[++nt] = a+b; break;
            case '-': num_stack[++nt] = a-b; break;
            case '*': num_stack[++nt] = a*b; break;
            case '/': num_stack[++nt] = a/b; break;
        }
    }

    // ... 遍历s, 数字入num_stack, 运算符按优先级处理
    return num_stack[0];
}
```

Q57: 双端队列(Deque)的应用?

 **秒懂**: 双端队列两头都能进出——既能当栈又能当队列用, 滑动窗口最大值问题就用单调双端队列, $O(n)$ 搞定原本 $O(nk)$ 的问题。

```
// 滑动窗口最大值(单调双端队列)
void maxSlidingWindow(int *nums, int n, int k, int *result) {
    int deque[n], front = 0, rear = -1; // 存下标
    int ri = 0;

    for (int i = 0; i < n; i++) {
        // 移除超出窗口的元素
        while (front <= rear && deque[front] <= i - k)
            front++;
        // 保持递减(移除比当前小的)
        while (front <= rear && nums[deque[rear]] <= nums[i])
            rear--;
        deque[++rear] = i;

        if (i >= k - 1)
            result[ri++] = nums[deque[front]];
    }
}
```

Q58: 栈实现递归转非递归?

 **秒懂：** 递归转非递归就是手动模拟系统栈——把递归参数压入自己的栈中循环处理，可以避免栈溢出风险，嵌入式中栈空间有限时必须会这招。

将递归DFS改为显式栈(嵌入式避免栈溢出):

```
// 递归版二叉树前序遍历
void preorder_recursive(TreeNode *root) {
    if (!root) return;
    visit(root);
    preorder_recursive(root->left);
    preorder_recursive(root->right);
}

// 非递归版(显式栈)
void preorder_iterative(TreeNode *root) {
    TreeNode *stack[100];
    int top = -1;
    if (root) stack[++top] = root;

    while (top >= 0) {
        TreeNode *node = stack[top--];
        visit(node);
        if (node->right) stack[++top] = node->right; // 右先入栈
        if (node->left) stack[++top] = node->left;   // 左后入栈(先出)
    }
}
```

Q59: 消息队列在RTOS中的实现?

 **秒懂：** RTOS消息队列就像邮箱——任务间通过队列传递固定大小的消息，发送方往里放，接收方取出处理，带阻塞超时机制实现任务同步和数据传递。

嵌入式中消息队列的底层实现:

```
typedef struct {
    uint8_t *buf;
    uint16_t item_size;
    uint16_t capacity;
    volatile uint16_t head;
    volatile uint16_t tail;
    volatile uint16_t count;
    // RTOS信号量
    SemaphoreHandle_t sem_space; // 可写空间
    SemaphoreHandle_t sem_data;  // 可读数据
} MsgQueue;

int mq_send(MsgQueue *q, void *msg, uint32_t timeout) {
    if (xSemaphoreTake(q->sem_space, timeout) != pdTRUE)
        return -1; // 超时
    memcpy(q->buf + q->head * q->item_size, msg, q->item_size);
```

```

q->head = (q->head + 1) % q->capacity;
xSemaphoreGive(q->sem_data); // 通知接收方
return 0;
}

```

Q60: 中缀转后缀表达式(逆波兰)?

 **秒懂：** 中缀转后缀用运算符栈——数字直接输出，运算符按优先级入栈或弹出，后缀表达式计算更简单（无括号无优先级），计算器和编译器都用这个。

编译器/计算器内部常用：

```

// "3+4*2" -> "3 4 2 * +"
void infix_to_postfix(const char *infix, char *postfix) {
    char op_stack[100]; int top = -1;
    int pi = 0;

    for (int i = 0; infix[i]; i++) {
        if (isdigit(infix[i])) {
            postfix[pi++] = infix[i];
        } else if (infix[i] == '(') {
            op_stack[++top] = '(';
        } else if (infix[i] == ')') {
            while (top >= 0 && op_stack[top] != '(')
                postfix[pi++] = op_stack[top--];
            top--; // 弹出 '('
        } else { // 运算符
            while (top >= 0 && priority(op_stack[top]) >= priority(infix[i]))
                postfix[pi++] = op_stack[top--];
            op_stack[++top] = infix[i];
        }
    }
    while (top >= 0) postfix[pi++] = op_stack[top--];
}

```

Q61: 栈和队列在BFS/DFS中的应用?

 **秒懂：** BFS用队列保证层层推进（像水波纹扩散），DFS用栈实现深入到底再回溯（像走迷宫走到死胡同再返回），两者是图/树遍历的两大基本方法。

DFS(深度优先)：用栈(或递归)

- 沿一条路走到底再回溯
- 适合：路径搜索/拓扑排序/连通分量

BFS(广度优先)：用队列

- 一层一层扩展
- 适合：最短路径/层次遍历/二分图判定

嵌入式场景：

DFS：状态机遍历/协议解析树

BFS：事件广播/网络拓扑发现

Q62: 环形队列判满判空？

💡 秒懂： 环形队列判满判空有三种方法——空一格法（ $\text{tail} + 1 == \text{head}$ 是满）、计数法（size变量）、标志位法（flag区分），最常用的是空一格法。

环形队列常见的两种方式：

```
// 方式1：浪费一个位置(最常用)
#define QUEUE_SIZE 16
typedef struct {
    int buf[QUEUE_SIZE];
    int head, tail;
} CircQueue;
// 空: head == tail
// 满: (tail + 1) % QUEUE_SIZE == head
// 长度: (tail - head + QUEUE_SIZE) % QUEUE_SIZE

// 方式2：用count计数
typedef struct {
    int buf[QUEUE_SIZE];
    int head, count;
} CircQueue2;
// 空: count == 0
// 满: count == QUEUE_SIZE
// tail = (head + count) % QUEUE_SIZE
```

Q63: 栈溢出检测(嵌入式)?

💡 秒懂： 嵌入式栈溢出就像水杯满了——RTOS每个任务栈独立，溢出会覆盖相邻内存导致诡异bug，检测方法：栈末尾填魔数、MPU保护、FreeRTOS栈水位线。

嵌入式中栈溢出是常见崩溃原因：

```
// FreeRTOS 栈溢出检测方法：
// 1. 任务创建时栈填充0xA5A5A5A5
// 2. 检测栈底的magic值是否被覆盖
configCHECK_FOR_STACK_OVERFLOW = 2 // 在FreeRTOSConfig.h中

void vApplicationStackOverflowHook(TaskHandle_t task, char *name) {
    printf("Stack overflow in task: %s\n", name);
    while(1); // 停下来调试
}

// 裸机方案：MPU保护栈边界
// 在栈底设置MPU只读区域 → 写入时触发MemManage异常
```


Q64: 无锁队列(Lock-Free)在嵌入式中的应用?

 **秒懂：** 无锁队列用CAS原子操作代替互斥锁——生产者消费者都通过原子比较交换操作来移动指针，避免加锁开销和优先级反转，适合ISR和任务间通信。

中断和主循环之间的无锁通信：

```
// 单生产者单消费者的无锁环形队列(ISR→主循环)
// 要点: head只有ISR写, tail只有主循环写
// 不需要锁! (前提: 32位对齐的读写是原子的)

typedef struct {
    volatile uint32_t head; // ISR写
    volatile uint32_t tail; // 主循环写
    uint8_t buf[256];      // 大小必须是2的幂
} LockFreeQueue;

// ISR中入队
void lfq_push(LockFreeQueue *q, uint8_t byte) {
    uint32_t next = (q->head + 1) & 0xFF;
    if (next != q->tail) {
        q->buf[q->head] = byte;
        q->head = next;
    }
}

// 主循环出队
int lfq_pop(LockFreeQueue *q, uint8_t *byte) {
    if (q->head == q->tail) return 0;
    *byte = q->buf[q->tail];
    q->tail = (q->tail + 1) & 0xFF;
    return 1;
}
```

四、树与堆 (Q65~Q85)

Q65: 二叉树的基本概念?

 **秒懂：** 二叉树——每个节点最多两个孩子（左和右），是最基本的树结构，像一个倒着长的树，从根到叶，面试中各种树问题都基于这个展开。

术语：

度：节点的子节点数量(二叉树最大为2)

深度：根到该节点的路径长度(根=0或1看定义)

高度：该节点到最远叶子的路径长度

特殊二叉树：

满二叉树：每层节点都是满的

完全二叉树：除最后一层外都满，最后一层左对齐

平衡二叉树：任何节点左右子树高度差 ≤ 1

性质:

第*i*层最多 $2^{(i-1)}$ 个节点

深度为*k*的树最多 $2^k - 1$ 个节点

叶子数 = 度2节点数 + 1

Q66: 二叉树遍历(前/中/后序)?

 **秒懂:** 前中后序就是根在前面、中间还是后面访问——前序（根左右）用于复制树，中序（左根右）用于BST排序输出，后序（左右根）用于释放内存。

```
typedef struct TreeNode {
    int val;
    struct TreeNode *left, *right;
} TreeNode;

// 前序: 根 → 左 → 右
void preorder(TreeNode *root) {
    if (!root) return;
    printf("%d ", root->val);
    preorder(root->left);
    preorder(root->right);
}

// 中序: 左 → 根 → 右 (BST中序遍历是有序的!)
void inorder(TreeNode *root) {
    if (!root) return;
    inorder(root->left);
    printf("%d ", root->val);
    inorder(root->right);
}

// 后序: 左 → 右 → 根
void postorder(TreeNode *root) {
    if (!root) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->val);
}
```

 **面试追问:** 快排最坏情况是什么? 怎么优化? 和归并排序怎么选?

 **嵌入式建议:** 嵌入式排序:数据量小(≤ 100)用插入排序(代码小);数据量大用堆排(原地/ $O(n\log n)$);qsort()是C标准库。

常用排序算法对比表

算法	时间(平均)	时间(最坏)	空间	稳定性	嵌入式适用
冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	✅	小数据量
插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	✅	★ 小/近有序

算法	时间(平均)	时间(最坏)	空间	稳定性	嵌入式适用
快速排序	$O(n\log n)$	$O(n^2)$	$O(\log n)$	✗	★通用首选
归并排序	$O(n\log n)$	$O(n\log n)$	$O(n)$	✓	需要稳定时
堆排序	$O(n\log n)$	$O(n\log n)$	$O(1)$	✗	大数据/TopK
计数排序	$O(n+k)$	$O(n+k)$	$O(k)$	✓	范围小的整数

💡 嵌入式首选: 小数组用插入排序, 通用场景用快排, 需要 $O(1)$ 空间用堆排

Q67: 层序遍历(BFS)?

💡 秒懂: 层序遍历用队列——先把根入队, 每次出队一个节点并把它的子节点入队, 就像广播一层一层地向下传播, 可以轻松求树的最大宽度和深度。

```
void levelOrder(TreeNode *root) {
    if (!root) return;
    TreeNode *queue[1000];
    int front = 0, rear = 0;
    queue[rear++] = root;

    while (front < rear) {
        int size = rear - front; // 当前层节点数
        for (int i = 0; i < size; i++) {
            TreeNode *node = queue[front++];
            printf("%d ", node->val);
            if (node->left) queue[rear++] = node->left;
            if (node->right) queue[rear++] = node->right;
        }
        printf("\n"); // 每层换行
    }
}
```

Q68: 二叉搜索树(BST)的操作?

💡 秒懂: BST左小右大——查找、插入都是 $O(\log n)$ 平均, 删除最复杂(找后继替代), 退化成链表就变 $O(n)$ 了, 所以才有AVL和红黑树来保持平衡。

```
// 查找:  $O(\log N)$ 平均,  $O(N)$ 最差
TreeNode* bst_search(TreeNode *root, int val) {
    while (root && root->val != val) {
        root = (val < root->val) ? root->left : root->right;
    }
    return root;
}

// 插入
TreeNode* bst_insert(TreeNode *root, int val) {
    if (!root) {
```

```

    TreeNode *node = malloc(sizeof(TreeNode));
    node->val = val; node->left = node->right = NULL;
    return node;
}
if (val < root->val) root->left = bst_insert(root->left, val);
else root->right = bst_insert(root->right, val);
return root;
}

// 删除(最复杂的操作)
// 三种情况: 叶子/一个子节点/两个子节点
// 两个子节点: 用右子树最小值(后继)替换

```

💡 **面试追问:** while(left<=right)和while(left<right)区别? mid怎么防溢出? 二分查找的变体?

🔧 **嵌入式建议:** 嵌入式查表/校准数据查找用二分。注意:数据必须有序;嵌入式Flash中的查找表用二分比遍历快得多。

Q69: 堆排序?

💡 **秒懂:** 堆排序用一棵'假装'的完全二叉树——在数组上用下标关系模拟父子节点, 建堆 $O(n)$, 每次取堆顶再调整 $O(\log n)$, 总共 $O(n\log n)$ 且原地排序。

```

void heapify(int *arr, int n, int i) {
    int largest = i;
    int left = 2*i + 1, right = 2*i + 2;
    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;
    if (largest != i) {
        int tmp = arr[i]; arr[i] = arr[largest]; arr[largest] = tmp;
        heapify(arr, n, largest);
    }
}

void heap_sort(int *arr, int n) {
    // 建堆:  $O(N)$ 
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    // 逐个取出最大值:  $O(N\log N)$ 
    for (int i = n-1; i > 0; i--) {
        int tmp = arr[0]; arr[0] = arr[i]; arr[i] = tmp;
        heapify(arr, i, 0);
    }
}

// 时间:  $O(N\log N)$ , 空间:  $O(1)$ , 不稳定

```

Q70: TopK问题(堆的应用)?

💡 **秒懂:** TopK问题用小顶堆——维护一个大小为K的小顶堆, 新元素比堆顶大就替换并调整, 最后堆里就是最大的K个元素, $O(n\log K)$ 比全排序快。

```

// 从N个数中找最大的K个: 用小顶堆(大小K)

```

```
// 如果新数>堆顶 → 替换堆顶并下沉
// 时间：O(NlogK)，比排序O(NlogN)更优当K<<N

void topK(int *nums, int n, int k, int *result) {
    MinHeap heap;
    heap_init(&heap);

    for (int i = 0; i < n; i++) {
        if (heap.size < k) {
            heap_push(&heap, nums[i]);
        } else if (nums[i] > heap_top(&heap)) {
            heap_pop(&heap);
            heap_push(&heap, nums[i]);
        }
    }
    // 堆中就是最大的K个数
    for (int i = 0; i < k; i++)
        result[i] = heap_pop(&heap);
}
```

Q71: AVL树和红黑树的区别？

 **秒懂：** AVL树严格平衡（左右子树高度差≤1）查找快但插入删除旋转多，红黑树近似平衡（用颜色规则）旋转少——Java的TreeMap、Linux的CFS调度器用红黑树。

特性	AVL树	红黑树
平衡条件	高度差≤1(严格)	黑色节点高度差≤1(松弛)
查找速度	略快(更平衡)	略慢
插入/删除	旋转次数多(O(logN))	旋转最多2~3次
适用场景	查找多的场景	插入删除频繁
典型使用	数据库索引	Linux进程调度/C++ map

Q72: 前缀树(Trie)?

 **秒懂：** Trie就像字典的目录结构——每个节点代表一个字符，从根到某节点的路径就是一个前缀，适合前缀匹配和自动补全，空间换时间。

用于字符串前缀匹配(如自动补全/路由表)：

```
/* 前缀树(Trie)? - 示例实现 */
#define ALPHABET_SIZE 26

typedef struct TrieNode { // 结构体定义
    struct TrieNode *children[ALPHABET_SIZE];
    int is_end;
} TrieNode;

void trie_insert(TrieNode *root, const char *word) {
```

```

TrieNode *cur = root;
while (*word) {
    int idx = *word - 'a';
    if (!cur->children[idx]) {
        cur->children[idx] = calloc(1, sizeof(TrieNode)); // 动态分配内存
    }
    cur = cur->children[idx];
    word++;
}
cur->is_end = 1;
}

int trie_search(TrieNode *root, const char *word) {
    TrieNode *cur = root;
    while (*word) {
        int idx = *word - 'a';
        if (!cur->children[idx]) return 0;
        cur = cur->children[idx];
        word++;
    }
    return cur->is_end;
}

```

Q73: 二叉树的最近公共祖先(LCA)?

 **秒懂：** LCA就是两个节点'回家路上'最先碰到的共同祖先——BST中利用大小关系查找，普通二叉树用递归从两头向上找交汇点。

```

TreeNode* lowestCommonAncestor(TreeNode *root, TreeNode *p, TreeNode *q) {
    if (!root || root == p || root == q) return root;
    TreeNode *left = lowestCommonAncestor(root->left, p, q);
    TreeNode *right = lowestCommonAncestor(root->right, p, q);
    if (left && right) return root; // p和q分别在左右子树
    return left ? left : right;
}
// 时间O(N)，空间O(H)(递归栈)

```

Q74: 二叉树的序列化和反序列化?

 **秒懂：** 序列化就是把树变成字符串（存储/传输），反序列化是从字符串重建树——通常用前序/层序遍历加null标记，面试常考。

```

// 前序遍历序列化(NULL用#表示)
void serialize(TreeNode *root, char *buf, int *pos) {
    if (!root) { buf[(*pos)++] = '#'; buf[(*pos)++] = ','; return; }
    *pos += sprintf(buf + *pos, "%d,", root->val);
    serialize(root->left, buf, pos);
    serialize(root->right, buf, pos);
}

// 反序列化

```

```

TreeNode* deserialize(char **ptr) {
    if (**ptr == '#') { (*ptr) += 2; return NULL; }
    int val = 0, sign = 1;
    if (**ptr == '-') { sign = -1; (*ptr)++; }
    while (**ptr != ',') { val = val * 10 + (**ptr - '0'); (*ptr)++; }
    (*ptr)++; // skip ','
    TreeNode *node = malloc(sizeof(TreeNode));
    node->val = sign * val;
    node->left = deserialize(ptr);
    node->right = deserialize(ptr);
    return node;
}

```

Q75: 完全二叉树的数组存储?

 **秒懂**: 完全二叉树用数组存——节点*i*的左孩子在 $2i+1$, 右孩子在 $2i+2$, 父亲在 $(i-1)/2$, 不浪费空间不用指针, 堆就是这么存的。

堆的底层实现方式:

```

// 完全二叉树可以用数组高效存储(无需指针)
// 下标从0开始:
//   父节点: (i-1)/2
//   左孩子: 2*i+1
//   右孩子: 2*i+2

// 应用: 堆、定时器管理(最小堆)、优先级调度
// 嵌入式优势: 无动态内存分配, 缓存友好

int tree[MAX_NODES];
int parent(int i) { return (i - 1) / 2; }
int left(int i) { return 2 * i + 1; }
int right(int i) { return 2 * i + 2; }

```

Q76: 判断平衡二叉树?

 **秒懂**: 判断平衡二叉树——递归求每个节点的左右子树高度差, 只要有一个节点高度差 >1 就不平衡, 自底向上 $O(n)$ 比自顶向下 $O(n^2)$ 高效。

```

int height(TreeNode *root) {
    if (!root) return 0;
    int lh = height(root->left);
    if (lh == -1) return -1;
    int rh = height(root->right);
    if (rh == -1) return -1;
    if (abs(lh - rh) > 1) return -1; // 不平衡
    return (lh > rh ? lh : rh) + 1;
}

int isBalanced(TreeNode *root) {
    return height(root) != -1;
}

```

// 时间 $O(N)$ ，一次遍历(自底向上)

Q77: 哈夫曼树(最优前缀编码)?

 **秒懂：** 哈夫曼树按频率建——频率低的在深层（长编码），频率高的在浅层（短编码），保证总编码长度最短，数据压缩的基础。

数据压缩的基础(嵌入式资源受限时有用):

构建方法：

1. 统计字符频率
2. 将所有字符作为叶子节点放入最小堆
3. 每次取出两个最小频率节点合并
4. 重复直到只剩一个根

例：A:5, B:9, C:12, D:13, E:16, F:45

编码：F=0, C=100, D=101, A=1100, B=1101, E=111

短编码给高频字符 → 总编码长度最短

嵌入式应用：固件压缩/传输数据压缩

Q78: B树和B+树?

 **秒懂：** B树每个节点能存多个key——像一本书的多级目录，减少磁盘IO次数。B+树数据全在叶子节点且叶子串起来，更适合数据库范围查询。

数据库索引和文件系统常用：

B树(m阶)：

- 每个节点最多m个子节点，m-1个关键字
- 所有叶子在同一层
- 查找可能在任何层结束

B+树：

- 数据只在叶子节点
- 叶子节点形成链表(范围查询高效)
- 内部节点只存索引(更多分支→更矮)

嵌入式应用：

- Flash文件系统(LittleFS用B树变体)
- 大容量数据索引(SD卡/eMMC)

Q79: 线段树(区间查询)?

 **秒懂：** 线段树把区间问题拆分到树节点上——每个节点存一段区间的信息（和/最大值等），查询和修改都是 $O(\log n)$ ，适合频繁区间查询的场景。

高效进行区间操作的数据结构：

```
int tree[4*MAX_N]; // 线段树数组(4倍空间)
```

```
// 建树
```



```

void build(int *arr, int node, int start, int end) {
    if (start == end) { tree[node] = arr[start]; return; }
    int mid = (start + end) / 2;
    build(arr, 2*node, start, mid);
    build(arr, 2*node+1, mid+1, end);
    tree[node] = tree[2*node] + tree[2*node+1]; // 维护区间和
}

// 区间查询[l,r]的和: O(logN)
int query(int node, int start, int end, int l, int r) {
    if (r < start || end < l) return 0;
    if (l <= start && end <= r) return tree[node];
    int mid = (start + end) / 2;
    return query(2*node, start, mid, l, r) +
           query(2*node+1, mid+1, end, l, r);
}

```

Q80: 并查集(Union-Find)?

 **秒懂：** 并查集就像找族长——合并时小家族挂到大家族下面，查找时路径压缩（直接指向族长），判断两人是否同族近似 $O(1)$ ，用于连通性问题。

解决连通性问题的结构：

```

int parent[MAX_N];
int rank_arr[MAX_N];

void uf_init(int n) {
    for (int i = 0; i < n; i++) { parent[i] = i; rank_arr[i] = 0; }
}

int uf_find(int x) { // 路径压缩
    if (parent[x] != x) parent[x] = uf_find(parent[x]);
    return parent[x];
}

void uf_union(int x, int y) { // 按秩合并
    int rx = uf_find(x), ry = uf_find(y);
    if (rx == ry) return;
    if (rank_arr[rx] < rank_arr[ry]) parent[rx] = ry;
    else if (rank_arr[rx] > rank_arr[ry]) parent[ry] = rx;
    else { parent[ry] = rx; rank_arr[rx]++; }
}

// 几乎O(1)的时间复杂度(阿克曼函数逆)

```

Q81: 二叉树的直径?

 **秒懂：** 二叉树直径是最长路径的边数——不一定经过根节点，递归求每个节点的左深度+右深度取最大值就是答案。

```

int diameter = 0;
int depth(TreeNode *root) {
    if (!root) return 0;
    int l = depth(root->left);
    int r = depth(root->right);
    diameter = (l + r > diameter) ? l + r : diameter;
    return (l > r ? l : r) + 1;
}
// diameter即为二叉树直径(最长路径经过的节点数-1)

```

Q82: 内存池的堆管理?

 **秒懂：** 内存池用堆管理空闲块——把大内存切成固定大小的块用空闲链表管理，分配O(1)，避免碎片化，嵌入式中比malloc+free可靠得多。

嵌入式中的固定大小内存池(避免碎片):

```

#define BLOCK_SIZE 32
#define POOL_SIZE 64

typedef struct { // 结构体定义
    uint8_t pool[POOL_SIZE][BLOCK_SIZE];
    uint8_t free_list[POOL_SIZE]; // 位图或链表索引
    int free_count;
} MemPool;

void* pool_alloc(MemPool *mp) {
    if (mp->free_count == 0) return NULL;
    for (int i = 0; i < POOL_SIZE; i++) {
        if (mp->free_list[i]) {
            mp->free_list[i] = 0;
            mp->free_count--;
            return mp->pool[i];
        }
    }
    return NULL;
}

void pool_free(MemPool *mp, void *ptr) {
    int idx = ((uint8_t*)ptr - (uint8_t*)mp->pool) / BLOCK_SIZE;
    mp->free_list[idx] = 1;
    mp->free_count++;
}

```

Q83: 字典树在网络路由中的应用?

 **秒懂：** 字典树在路由中做最长前缀匹配——IP地址按位插入Trie，查找时尽可能走深，最后匹配到的节点对应的路由就是目标。

IP路由表的最长前缀匹配:

路由表(二进制前缀):

192.168.1.0/24 → 接口1
192.168.0.0/16 → 接口2
0.0.0.0/0 → 默认网关

用Trie(按位):

从IP高位到低位查找

每次匹配到一个路由就记录(可能更长前缀继续匹配)

最后返回最长匹配的路由

嵌入式场景: 1wIP的路由表查找

Q84: 树的Morris遍历(O(1)空间)?

 **秒懂:** Morris遍历利用叶子节点的空指针——临时连成线索回到祖先节点, 省掉了栈或递归的O(n)空间, 但会临时修改树结构(遍历完恢复)。

不用栈/递归实现中序遍历(空间O(1)):

```
void morris_inorder(TreeNode *root) {
    TreeNode *cur = root;
    while (cur) {
        if (!cur->left) {
            printf("%d ", cur->val);
            cur = cur->right;
        } else {
            // 找前驱(左子树最右节点)
            TreeNode *pred = cur->left;
            while (pred->right && pred->right != cur)
                pred = pred->right;

            if (!pred->right) {
                pred->right = cur; // 建线索
                cur = cur->left;
            } else {
                pred->right = NULL; // 恢复
                printf("%d ", cur->val);
                cur = cur->right;
            }
        }
    }
}
```

// 适合嵌入式: 栈空间极度受限时

Q85: 堆在定时器管理中的应用?

 **秒懂:** 定时器用最小堆管理——堆顶永远是最快到期的定时器, 检查时只看堆顶O(1), 插入和删除O(logn), 比链表遍历O(n)高效得多。

// 最小堆管理多个定时器(最近超时的在堆顶)

```
typedef struct {
```

```

uint32_t expire_tick; // 超时时刻
void (*callback)(void *);
void *arg;
} Timer;

Timer timer_heap[MAX_TIMERS];
int timer_count = 0;

// 主循环检查
void timer_check(uint32_t now) {
    while (timer_count > 0 && timer_heap[0].expire_tick <= now) {
        Timer t = heap_extract_min(timer_heap, &timer_count);
        t.callback(t.arg);
    }
}

// O(logN)插入/删除, O(1)检查最近超时

```

五、哈希表（Q86~Q91）

Q86: 哈希表的基本原理？

 **秒懂：** 哈希表就像按身份证号查人——key经过哈希函数算出一个下标直接存取，平均O(1)，是查找速度最快的数据结构。

核心：key → hash(key) → index → value

哈希函数：

整数：hash = key % table_size (table_size取素数更好)

字符串：hash = sum(s[i] * 31ⁱ) % table_size

冲突解决：

1. 链地址法(拉链)：每个位置挂链表
2. 开放寻址法：冲突后找下一个空位
 - 线性探测：h+1, h+2, h+3...
 - 二次探测：h+1, h+4, h+9...
 - 双重哈希：h + i*hash2(key)

Q87: 哈希表的C实现(链地址法)？

 **秒懂：** 链地址法就是每个哈希桶挂一条链表——冲突了就追加到链表上，简单但链表长了查找变慢，可以设置负载因子触发扩容。

```

#define TABLE_SIZE 256

typedef struct HashNode { // 结构体定义
    char *key;
    int value;
    struct HashNode *next;
} HashNode;

```

```

typedef struct { HashNode *buckets[TABLE_SIZE]; } HashMap; // 结构体定义

unsigned int hash(const char *key) {
    unsigned int h = 0;
    while (*key) h = h * 31 + *key++;
    return h % TABLE_SIZE;
}

void hashmap_put(HashMap *map, const char *key, int value) {
    unsigned int idx = hash(key);
    HashNode *node = map->buckets[idx];
    while (node) {
        if (strcmp(node->key, key) == 0) { node->value = value; return; }
        node = node->next;
    }
    HashNode *new_node = malloc(sizeof(HashNode)); // 动态分配内存
    new_node->key = strdup(key);
    new_node->value = value;
    new_node->next = map->buckets[idx];
    map->buckets[idx] = new_node;
}

int hashmap_get(HashMap *map, const char *key, int *value) {
    unsigned int idx = hash(key);
    HashNode *node = map->buckets[idx];
    while (node) {
        if (strcmp(node->key, key) == 0) { *value = node->value; return 1; }
        node = node->next;
    }
    return 0; // not found
}

```

Q88: 两数之和(哈希表经典应用)?

 **秒懂：** 两数之和用哈希表——遍历数组每个数x，查哈希表里有没有target-x，有就找到了，没有就把x存进去，一次遍历O(n)搞定。

```

// O(N)解法：遍历数组，对每个num查找target-num是否在哈希表中
typedef struct { int key; int idx; UT_hash_handle hh; } HashEntry;

int* twoSum(int *nums, int n, int target) {
    HashEntry *map = NULL;
    static int result[2];

    for (int i = 0; i < n; i++) {
        int complement = target - nums[i];
        HashEntry *found;
        HASH_FIND_INT(map, &complement, found);
        if (found) {
            result[0] = found->idx; result[1] = i;
            return result;
        }
    }
}

```

```

    HashEntry *entry = malloc(sizeof(HashEntry)); // 动态分配内存
    entry->key = nums[i]; entry->idx = i;
    HASH_ADD_INT(map, key, entry);
}
return NULL;
}

```

Q89: 布隆过滤器(Bloom Filter)?

 **秒懂：** 布隆过滤器用多个哈希函数和位数组——说'不存在'一定不存在，说'存在'可能误判，适合缓存穿透、爬虫URL去重等容忍小概率误判的场景。

空间高效的概率型数据结构(嵌入式IoT常用)：

```

// 判断元素"可能存在"或"一定不存在"
#define BF_SIZE 1024 // 位数组大小(位)
#define NUM_HASH 3

uint8_t bf_bits[BF_SIZE / 8];

void bf_add(const char *key) {
    for (int i = 0; i < NUM_HASH; i++) {
        unsigned int h = hash_func(key, i) % BF_SIZE;
        bf_bits[h / 8] |= (1 << (h % 8));
    }
}

int bf_check(const char *key) {
    for (int i = 0; i < NUM_HASH; i++) {
        unsigned int h = hash_func(key, i) % BF_SIZE;
        if (!(bf_bits[h / 8] & (1 << (h % 8))))
            return 0; // 一定不存在
    }
    return 1; // 可能存在(有误判率)
}

// 应用：网络爬虫URL去重/缓存过滤/IoT设备白名单

```

Q90: 一致性哈希(分布式系统)?

 **秒懂：** 一致性哈希把服务器和数据映射到一个环上——增减服务器只影响相邻节点的数据，不会引起全局数据迁移，分布式系统解决扩缩容问题的利器。

了解即可(面试可能问原理)：

传统哈希：hash(key) % N → 增减节点需要大量数据迁移
 一致性哈希：将哈希空间组织成环

- 节点映射到环上
- key顺时针找最近节点
- 增删节点只影响相邻区域(约1/N的数据迁移)

嵌入式相关：分布式IoT网关的负载均衡

Q91: 哈希表在嵌入式中的应用？

 **秒懂：** 嵌入式端哈希表要注意——内存受限用静态数组而非动态分配，哈希函数要简单高效（如取模），用于注册表、命令解析、设备查找等场景。

1. 命令表：
hash("LED_ON") → LED_ON_handler
hash("TEMP_READ") → TEMP_READ_handler
O(1)命令分发(比if-else链快)
2. 事件订阅：
hash(EVENT_ID) → subscriber_list
快速找到事件的所有订阅者
3. 缓存(LRU)：
hash(key) → cached_value
快速判断数据是否已缓存
4. 去重：
DMA接收的CAN帧 → hash(ID+data) → 丢弃重复帧

六、排序算法（Q92~Q102）

-  **面试追问：** LRU的时间复杂度？用什么数据结构实现O(1)?LinkedHashMap了解吗？
-  **嵌入式建议：** 嵌入式Cache管理/DNS缓存/资源池都用LRU思想。手写LRU是面试超高频题。

Q92: 常见排序算法对比？

 **秒懂：** 排序算法对比就像工具箱——冒泡O(n²)但简单、快排平均O(nlogn)最常用、归并O(nlogn)稳定但要额外空间、堆排原地但不稳定。

算法	平均	最差	空间	稳定	特点
冒泡	O(N²)	O(N²)	O(1)	✓	简单,几乎不用
选择	O(N²)	O(N²)	O(1)	✗	交换次数少
插入	O(N²)	O(N²)	O(1)	✓	小数据/近有序很快
快排	O(NlogN)	O(N²)	O(logN)	✗	通常最快
归并	O(NlogN)	O(NlogN)	O(N)	✓	稳定+确定性
堆排	O(NlogN)	O(NlogN)	O(1)	✗	原地+确定性
计数	O(N+K)	O(N+K)	O(K)	✓	整数范围小时

Q93: 快速排序的实现?

💡 秒懂：快排——选基准把数组分成两半（小的左边大的右边），递归排两半，分治思想，平均 $O(n\log n)$ 最快，但最坏 $O(n^2)$ （已排序时）。

```
void quick_sort(int *arr, int left, int right) {
    if (left >= right) return;
    int pivot = arr[left + (right-left)/2]; // 中间作为枢轴
    int i = left, j = right;
    while (i <= j) {
        while (arr[i] < pivot) i++;
        while (arr[j] > pivot) j--;
        if (i <= j) {
            int tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
            i++; j--;
        }
    }
    quick_sort(arr, left, j);
    quick_sort(arr, i, right);
}
// 平均 $O(N\log N)$ ，最差 $O(N^2)$ （已有序数组+首元素做枢轴）
```

全网最详细的嵌入式实习/秋招/春招公司清单，正在更新中，需要可关注微信公众号：嵌入式校招菌。

Q94: 归并排序的实现?

💡 秒懂：归并排序——先分成两半各自排好，再合并两个有序数组，稳定且一定是 $O(n\log n)$ ，缺点是需要 $O(n)$ 额外空间，链表排序的首选。

```
void merge(int *arr, int *tmp, int left, int mid, int right) {
    int i = left, j = mid + 1, k = left;
    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) tmp[k++] = arr[i++];
        else tmp[k++] = arr[j++];
    }
    while (i <= mid) tmp[k++] = arr[i++];
    while (j <= right) tmp[k++] = arr[j++];
    memcpy(arr + left, tmp + left, (right - left + 1) * sizeof(int)); // 内存拷贝
}

void merge_sort(int *arr, int *tmp, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    merge_sort(arr, tmp, left, mid);
    merge_sort(arr, tmp, mid + 1, right);
    merge(arr, tmp, left, mid, right);
}
// 稳定 +  $O(N\log N)$ 确定性 + 但需要 $O(N)$ 额外空间
```


Q95: 插入排序(小数据最优)?

 **秒懂：** 插入排序就像打牌时理牌——每次拿一张插到已排好序的手牌中正确位置，小数据量时比快排快（少了递归开销），STL的sort对小数组就用插入排序。

```
void insertion_sort(int *arr, int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i], j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
}
// 最好O(N)(已有序)，最差O(N²)
// 嵌入式优势：代码短，N<16时比快排快(无递归开销)
```

Q96: 嵌入式中排序算法的选择?

 **秒懂：** 嵌入式选排序看场景——数据量小（<50）用插入排序最快，中等规模用快排，需要稳定性用归并，内存紧张用堆排，几乎有序用冒泡/插入。

N < 16：插入排序(代码小,无递归,近有序很快)
N中等：快排(一般最快)或堆排(确定性+无额外内存)
需要稳定：归并排序(但需要O(N)额外空间)
内存极度受限：堆排序(O(1)额外空间)
已知数据范围小：计数排序(O(N))

实际应用：

- qsort标准库：通常混合快排+插入排序
- RTOS任务优先级：维护有序链表(插入排序思想)
- 传感器采样中值滤波：小数组排序(插入排序)

Q97: 第K大的元素(快速选择)?

 **秒懂：** 快速选择（QuickSelect）——和快排一样做partition，但只递归目标所在的那一半，平均O(n)找到第K大，不需要完整排序。

```
// 基于快排partition,平均O(N)
int partition(int *arr, int left, int right) {
    int pivot = arr[right], i = left;
    for (int j = left; j < right; j++) {
        if (arr[j] <= pivot) {
            int tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
            i++;
        }
    }
    int tmp = arr[i]; arr[i] = arr[right]; arr[right] = tmp;
    return i;
}
```

```
int quickSelect(int *arr, int left, int right, int k) {
    int pivot_idx = partition(arr, left, right);
    if (pivot_idx == k) return arr[k];
    else if (pivot_idx < k) return quickSelect(arr, pivot_idx+1, right, k);
    else return quickSelect(arr, left, pivot_idx-1, k);
}
// 平均O(N), 最差O(N^2)
```

Q98: 外部排序(大数据)?

 **秒懂**: 外部排序——数据太大放不进内存，分批读入排序写回磁盘，然后多路归并合并有序块，数据库和大数据处理中常用。

数据量超过内存时的排序方法：

步骤：

1. 分块：将大文件分成多个能放入内存的块
2. 内排：每块在内存中排序后写回磁盘
3. 多路归并：从每个排好序的块各取一个元素
 - 放入最小堆 → 取出最小 → 输出
 - 从该块再取一个放入堆

嵌入式场景：

- 大量日志排序 (Flash→RAM→Flash)
- SD卡中的大文件排序

优化：K路归并用最小堆→ $O(N\log K)$

Q99: 基数排序(非比较排序)?

 **秒懂**: 基数排序——从最低位到最高位逐位排序（每位用计数排序），不比较大小， $O(d \times n)$ ，适合整数和定长字符串排序。

```
// 按位排序, 从低位到高位(用计数排序做稳定分配)
void radix_sort(int *arr, int n) {
    int max_val = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > max_val) max_val = arr[i];

    int *output = malloc(n * sizeof(int)); // 动态分配内存
    for (int exp = 1; max_val / exp > 0; exp *= 10) {
        int count[10] = {0};
        for (int i = 0; i < n; i++)
            count[(arr[i] / exp) % 10]++;
        for (int i = 1; i < 10; i++)
            count[i] += count[i-1];
        for (int i = n-1; i >= 0; i--) {
            int digit = (arr[i] / exp) % 10;
            output[--count[digit]] = arr[i];
        }
        memcpy(arr, output, n * sizeof(int)); // 内存拷贝
    }
}
```

```

    }
    free(output); // 释放内存(建议之后置NULL)
}
// O(d*(N+K)), d=位数, K=基数(10)

```

Q100: 稳定排序的意义?

 **秒懂**: 稳定排序保证相等元素的相对顺序不变——冒泡、插入、归并是稳定的，快排、堆排不稳定，需要保持原有顺序时必须用稳定排序。

稳定: 相等元素排序后相对顺序不变
 不稳定: 可能改变相等元素的相对顺序

重要场景:

1. 多关键字排序:
 先按时间排序(稳定) → 再按优先级排序(稳定)
 → 同优先级的仍按时间有序!
2. 结构体排序:
 两个传感器数据值相同 → 保持原始采集顺序

嵌入式: 通常数据量小, 稳定性不那么重要
 但如果需要稳定 → 选归并/插入排序

Q101: 排序算法的稳定性验证?

 **秒懂**: 验证排序稳定性——给相等元素加编号排序后检查编号顺序，如'3a 3b'排完还是'3a 3b'就是稳定的，变成'3b 3a'就不稳定。

如何验证一个排序实现是否稳定:

```

// 方法: 给相等元素加序号, 排序后检查序号顺序
typedef struct { int val; int orig_idx; } Item;

int is_stable_sort(void (*sort_func)(Item*, int), Item *arr, int n) {
    sort_func(arr, n);
    for (int i = 1; i < n; i++) {
        if (arr[i].val == arr[i-1].val) {
            if (arr[i].orig_idx < arr[i-1].orig_idx)
                return 0; // 不稳定!
        }
    }
    return 1; // 稳定
}

```

Q102: 冒泡排序的优化?

 **秒懂**: 冒泡优化——加一个flag，如果某趟没有发生交换说明已经有序了直接结束，最好情况从 $O(n^2)$ 优化到 $O(n)$ 。

// 优化1: 提前终止(某趟无交换 → 已有序)

```

void bubble_sort_opt(int *arr, int n) {
    for (int i = 0; i < n-1; i++) {
        int swapped = 0;
        for (int j = 0; j < n-1-i; j++) {
            if (arr[j] > arr[j+1]) {
                int tmp = arr[j]; arr[j] = arr[j+1]; arr[j+1] = tmp;
                swapped = 1;
            }
        }
        if (!swapped) break; // 已有序
    }
}
// 最好O(N) (已有序时第一趟无交换就退出)

```

七、图与高级算法 (Q103~Q120)

Q103: 图的表示方法?

 **秒懂：** 图有两种存储方式——邻接矩阵（二维数组，适合稠密图，O(1)查询）和邻接表（链表数组，适合稀疏图，省空间），嵌入式中后者更常用。

```

// 1. 邻接矩阵(适合稠密图)
int graph[N][N]; // graph[i][j]=权重, 0表示无边
// 空间O(N²), 查询O(1), 遍历邻居O(N)

// 2. 邻接表(适合稀疏图, 嵌入式常用)
typedef struct Edge { int to; int weight; struct Edge *next; } Edge;
Edge *adj[N]; // adj[i]指向i的所有邻居链表
// 空间O(N+E), 遍历邻居O(度)

```

Q104: BFS(广度优先搜索)?

 **秒懂：** BFS就像往池塘扔石头——从起点一圈一圈向外扩展，用队列保证层层推进，能找最短路径，适合无权图的最短距离。

```

void bfs(int graph[][N], int n, int start) {
    int visited[N] = {0};
    int queue[N], front = 0, rear = 0;

    visited[start] = 1;
    queue[rear++] = start;

    while (front < rear) {
        int node = queue[front++];
        printf("%d ", node);
        for (int i = 0; i < n; i++) {
            if (graph[node][i] && !visited[i]) {
                visited[i] = 1;
                queue[rear++] = i;
            }
        }
    }
}

```

```

    }
}
// 应用：最短路径(无权图)、层次遍历、连通性检测

```

Q105: DFS(深度优先搜索)?

 **秒懂：** DFS就像走迷宫——沿一条路走到底，走不通就回退换一条，用栈（或递归）实现，适合连通性判断、拓扑排序、路径搜索。

```

void dfs(int graph[][N], int n, int node, int *visited) {
    visited[node] = 1;
    printf("%d ", node);
    for (int i = 0; i < n; i++) {
        if (graph[node][i] && !visited[i]) {
            dfs(graph, n, i, visited);
        }
    }
}
// 应用：拓扑排序、连通分量、路径搜索、回溯算法

```

Q106: Dijkstra最短路径?

 **秒懂：** Dijkstra就像修路工从起点往外修——每次找距离最小的未访问节点，更新其邻居的最短距离，用最小堆 $O((V+E)\log V)$ ，不能处理负权边。

```

// 单源最短路径(权重非负)
void dijkstra(int graph[][N], int n, int src, int *dist) {
    int visited[N] = {0};
    for (int i = 0; i < n; i++) dist[i] = INT_MAX;
    dist[src] = 0;

    for (int count = 0; count < n; count++) {
        // 找未访问的最小距离节点
        int u = -1, min_dist = INT_MAX;
        for (int i = 0; i < n; i++)
            if (!visited[i] && dist[i] < min_dist) { u = i; min_dist = dist[i]; }
        if (u == -1) break;
        visited[u] = 1;

        // 更新邻居
        for (int v = 0; v < n; v++) {
            if (graph[u][v] && !visited[v] && dist[u] + graph[u][v] < dist[v])
                dist[v] = dist[u] + graph[u][v];
        }
    }
}
// O(N^2)简单版，用堆优化可达O((N+E)logN)

```

Q107: 拓扑排序?

 **秒懂：** 拓扑排序就像排课程先后顺序——必须先修完先修课才能选后续课，用入度法（BFS）或DFS后序反转，结果不唯一但必须是DAG。

```
// 有向无环图(DAG)的线性排序(依赖关系)
// BFS方法(Kahn算法):
void topological_sort(int graph[][N], int n) {
    int in_degree[N] = {0};
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (graph[i][j]) in_degree[j]++;

    int queue[N], front = 0, rear = 0;
    for (int i = 0; i < n; i++)
        if (in_degree[i] == 0) queue[rear++] = i;

    while (front < rear) {
        int node = queue[front++];
        printf("%d ", node);
        for (int i = 0; i < n; i++) {
            if (graph[node][i]) {
                if (--in_degree[i] == 0) queue[rear++] = i;
            }
        }
    }
}

// 应用：任务调度/编译依赖/初始化顺序
```

Q108: 最小生成树(Kruskal)?

 **秒懂：** Kruskal最小生成树——把所有边按权重排序，从小到大选边（用并查集判断是否成环），选n-1条边就完成，贪心思想。

```
// 按边权排序 + 并查集检测环
typedef struct { int u, v, w; } Edge;

int cmp_edge(const void *a, const void *b) {
    return ((Edge*)a)->w - ((Edge*)b)->w;
}

int kruskal(Edge *edges, int edge_count, int n) {
    qsort(edges, edge_count, sizeof(Edge), cmp_edge);
    uf_init(n);
    int mst_weight = 0, count = 0;

    for (int i = 0; i < edge_count && count < n-1; i++) {
        if (uf_find(edges[i].u) != uf_find(edges[i].v)) {
            uf_union(edges[i].u, edges[i].v);
            mst_weight += edges[i].w;
            count++;
        }
    }
}
```

```
}  
    return mst_weight;  
}
```

Q109: 动态规划基础(背包问题)?

 **秒懂：** 动态规划就像'记账本'——把大问题拆成子问题，子问题的结果记录下来避免重复计算，背包问题是经典入门：n个物品有重量和价值，背包容量有限，求最大价值。

```
// 0-1背包：N个物品，容量w，求最大价值  
int knapsack(int *weight, int *value, int n, int w) {  
    int dp[w+1];  
    memset(dp, 0, sizeof(dp));  
  
    for (int i = 0; i < n; i++) {  
        for (int w = w; w >= weight[i]; w--) { // 逆序!  
            if (dp[w-weight[i]] + value[i] > dp[w])  
                dp[w] = dp[w-weight[i]] + value[i];  
        }  
    }  
    return dp[w];  
}  
// 空间优化到O(w)，时间O(NW)
```

Q110: A*算法(启发式搜索)?

 **秒懂：** A*算法就像有导航的路径搜索——在Dijkstra基础上加了启发函数（估计到终点的距离），优先搜索更可能离终点近的节点，机器人路径规划常用。

路径规划算法(机器人/游戏常用):

A*: $f(n) = g(n) + h(n)$

$g(n)$: 起点到n的实际代价

$h(n)$: n到终点的估计代价(启发函数)

用优先级队列(按f值排序):

1. 将起点加入open_list
2. 取f最小的节点展开
3. 对每个邻居: 计算g和h, 加入open_list
4. 直到到达终点

启发函数选择:

曼哈顿距离: $|x1-x2| + |y1-y2|$ (4方向移动)

欧几里得距离: $\sqrt{(x1-x2)^2 + (y1-y2)^2}$ (任意方向)

嵌入式应用: 小车路径规划、扫地机器人

Q111: KMP字符串匹配?

 **秒懂：** KMP在这里再提一次是因为字符串匹配太重要——核心是next数组（部分匹配表），匹配失败时模式串不从头开始而是跳到合适位置，面试必掌握。

```
// 构建next数组(部分匹配表)
void buildNext(const char *pattern, int *next, int m) {
    next[0] = -1;
    int i = 0, j = -1;
    while (i < m) {
        if (j == -1 || pattern[i] == pattern[j]) {
            i++; j++; next[i] = j;
        } else {
            j = next[j];
        }
    }
}

int kmp_search(const char *text, const char *pattern) {
    int n = strlen(text), m = strlen(pattern);
    int next[m+1];
    buildNext(pattern, next, m);

    int i = 0, j = 0;
    while (i < n) {
        if (j == -1 || text[i] == pattern[j]) { i++; j++; }
        else j = next[j];
        if (j == m) return i - m; // 匹配成功
    }
    return -1;
}
// 时间O(N+M)，比暴力O(NM)快
```

Q112: 位运算技巧?

 **秒懂：** 位运算技巧就像程序员的'魔术'—— $x \& (x-1)$ 去掉最低位的1、 $x^x=0$ 、 $n \& 1$ 判断奇偶、 $1 \leq k$ 设置第k位，嵌入式操作寄存器天天用。

嵌入式面试高频(寄存器操作基础):

```
// 1. 判断奇偶
n & 1 // 1=奇, 0=偶

// 2. 交换两数(不用临时变量)
a ^= b; b ^= a; a ^= b;

// 3. 取最低位的1
n & (-n) // lowbit

// 4. 清除最低位的1
n & (n - 1)
```



```
// 5. 统计1的个数(Brian Kernighan)
int count = 0;
while (n) { n &= (n-1); count++; }

// 6. 判断是否是2的幂
(n > 0) && (n & (n-1)) == 0

// 7. 设置/清除/翻转第k位
n | (1<<k)    // 设置
n & ~(1<<k)   // 清除
n ^ (1<<k)    // 翻转
```

💡 **面试追问：** 环形缓冲区怎么判断满和空？用size计数还是浪费一个格子？

🔧 **嵌入式建议：** UART/ADC/音频数据接收必用环形缓冲区。推荐size计数法(清晰);大小用2的幂(可以用位与代替取模)。

Q113: 位图(Bitmap)在嵌入式中的应用？

💡 **秒懂：** 位图用一个bit表示一个状态——32位int可以管理32个标志位，嵌入式中用于GPIO状态、任务标志、中断标志、RTOS事件组，极省空间。

```
// 用1位表示一个状态(极省空间)
#define BITMAP_SIZE 1024
uint32_t bitmap[BITMAP_SIZE / 32];

void bitmap_set(int n) { bitmap[n/32] |= (1u << (n%32)); }
void bitmap_clear(int n) { bitmap[n/32] &= ~(1u << (n%32)); }
int bitmap_test(int n) { return (bitmap[n/32] >> (n%32)) & 1; }

// 应用：
// 1. 内存块分配管理(FreeRTOS堆管理)
// 2. IO引脚状态记录
// 3. 事件标志组(EventGroup)
// 4. 权限位/状态机标志
```

Q114: 环形缓冲区的位操作优化？

💡 **秒懂：** 环形缓冲区大小设为2的幂——取模运算变成与运算 (index & (size-1))，省掉除法操作，在MCU上速度提升明显。

```
// 当size是2的幂时，用位与代替取模
#define BUF_SIZE 256 // 必须是2的幂
#define BUF_MASK (BUF_SIZE - 1)

uint8_t buf[BUF_SIZE];
uint32_t head = 0, tail = 0;

void push(uint8_t data) {
    buf[head & BUF_MASK] = data; // head & 0xFF 代替 head % 256
    head++;
}
```

```
uint8_t pop(void) {
    uint8_t data = buf[tail & BUF_MASK];
    tail++;
    return data;
}

// 可用数据量: head - tail (利用无符号溢出自动处理)
// 注意: head/tail不取模,让它们自然溢出 → 简化逻辑
```

Q115: 状态机在嵌入式中的数据结构实现?

 **秒懂:** 状态机用枚举+switch或函数指针表实现——每个状态是一个枚举值或函数,事件触发状态转移,嵌入式协议解析、按键处理、设备控制都靠状态机。

```
// 表驱动状态机(查表法)
typedef enum { ST_IDLE, ST_RUNNING, ST_ERROR, ST_COUNT } State;
typedef enum { EV_START, EV_STOP, EV_FAULT, EV_RESET, EV_COUNT } Event;

typedef void (*Action)(void);
typedef struct { State next_state; Action action; } Transition;

Transition table[ST_COUNT][EV_COUNT] = {
    /*           EV_START           EV_STOP           EV_FAULT           EV_RESET */
    [ST_IDLE]    = {{ST_RUNNING, do_start}, {ST_IDLE, NULL}, {ST_ERROR, do_alarm}, {ST_IDLE,
    NULL}},
    [ST_RUNNING] = {{ST_RUNNING, NULL}, {ST_IDLE, do_stop}, {ST_ERROR, do_alarm}, {ST_IDLE,
    do_reset}},
    [ST_ERROR]   = {{ST_ERROR, NULL}, {ST_ERROR, NULL}, {ST_ERROR, NULL}, {ST_IDLE,
    do_reset}},
};

void fsm_dispatch(State *state, Event event) {
    Transition *t = &table[*state][event];
    if (t->action) t->action();
    *state = t->next_state;
}
```

Q116: 双向链表在内核中的应用(Linux list_head)?

 **秒懂:** Linux list_head是侵入式双向链表——结构体里嵌入链表节点而不是链表节点里放数据指针,通过container_of宏反推结构体地址,内核到处都用。

```
// Linux内核的通用双向链表
struct list_head {
    struct list_head *next, *prev;
};

// 嵌入到任意结构体中
struct task_struct {
    int pid;
```

```

    struct list_head list; // 链入就绪队列
};

// container_of: 从list_head指针反推结构体指针
#define container_of(ptr, type, member) \
    ((type *)((char *)(ptr) - offsetof(type, member)))

// 遍历
#define list_for_each_entry(pos, head, member) \
    for (pos = container_of((head)->next, typeof(*pos), member); \
        &pos->member != (head); \
        pos = container_of(pos->member.next, typeof(*pos), member))

```

Q117: CRC校验的数据结构实现?

 **秒懂：** CRC校验就像给数据算'指纹'——选定多项式后对数据做模2除法，余数就是CRC值，接收端重新算一次比对，不一致说明传输出错。

```

// CRC-16查表法(空间换时间)
static const uint16_t crc16_table[256] = {
    0x0000, 0xc0c1, 0xc181, 0x0140, /* ... 256 entries ... */
};

uint16_t crc16(const uint8_t *data, uint16_t len) {
    uint16_t crc = 0xffff;
    while (len--) {
        crc = (crc >> 8) ^ crc16_table[(crc ^ *data++) & 0xff];
    }
    return crc;
}

// 嵌入式选择:
// 查表法: 速度快, 占256x2=512字节ROM
// 逐位计算: 速度慢, 不占额外空间
// 折中: 半字节查表(16项表)

```

Q118: 内存对齐和结构体填充?

 **秒懂：** 内存对齐就像停车位——变量地址要对齐到其类型大小的倍数（4字节int对齐到4的倍数），结构体可能有空洞（padding），用sizeof确认实际大小。

```

// 面试高频: 下面结构体占多少字节?
struct A {
    char a;    // offset 0, size 1
    // padding 3 bytes
    int b;     // offset 4, size 4
    char c;    // offset 8, size 1
    // padding 3 bytes (总大小对齐到最大成员=4)
}; // sizeof = 12

struct B {

```

```

char a;    // offset 0
char c;    // offset 1
// padding 2 bytes
int b;     // offset 4
}; // sizeof = 8 (调整顺序可以省空间!)

// 嵌入式通信中: __attribute__((packed)) 取消填充
struct __attribute__((packed)) Packet {
    uint8_t type;
    uint32_t data; // 非对齐访问!某些CPU会fault
}; // sizeof = 5

```

Q119: 软件定时器的数据结构选择?

 **秒懂**: 软件定时器用最小堆效率最高——堆顶是最快到期的定时器 $O(1)$ 检查, 增删 $O(\log n)$ 。时间轮 (TimeWheel) 适合大量定时器, FreeRTOS用链表。

需求: 管理大量定时器, 支持高效的插入/删除/查询最近超时

方案对比:

- 有序链表: 插入 $O(N)$, 查询最近 $O(1)$, 适合定时器少
- 最小堆: 插入 $O(\log N)$, 查询 $O(1)$, 删除 $O(\log N)$, 通用方案
- 时间轮: 插入 $O(1)$, 推进 $O(1)$, 适合大量短定时器

时间轮 (Timer wheel):

- 类似钟表:
 - 256格的环形数组 (每格一个链表)
 - tick++时推进到下一格, 执行该格所有定时器
 - 超出一圈的放到更高层

Linux内核: 分层时间轮

FreeRTOS: 有序链表 (定时器数量通常不多)

Q120: 数据结构在嵌入式面试中的考察重点?

 **秒懂**: 数据结构面试考察重点——链表反转+环检测必考, 栈队列基本操作必会, 排序快排+归并手写, 树的遍历+BST, 嵌入式还要会环形缓冲区和位操作。

高频考点:

1. 链表操作 (翻转/环/合并) - 指针操作能力
2. 栈/队列 (用数组实现) - 基本功
3. 环形缓冲区 - 串口/DMA必备
4. 排序 (快排/插入/堆排) - 算法基础
5. 哈希表 - 查找效率
6. 堆 (优先级队列) - RTOS调度/定时器
7. 位操作 - 寄存器/标志位
8. 状态机 - 协议解析/控制逻辑
9. 树 (尤其完全二叉树/堆) - 数据组织
10. 内存对齐/结构体 - 通信协议

面试建议:

- 手写代码(白板),重点是指针和边界条件
- 分析时间复杂度和空间复杂度
- 说明嵌入式场景下的选择理由(内存有限/无动态分配)

本文档由公众号：**嵌入式校招菌** 原创整理，欢迎关注获取更多嵌入式校招信息

📌 **嵌入式校招excel** 全年更新中，实习/秋招/春招都包含，纯人工维护，已支持24/25/26届同学毕业，减少招聘信息差，你没听过的公司只要有嵌入式岗位我都会收集，一次付费永久有效，更有嵌入式微信/飞书交流群；用过的同学都说好！

需要的可以关注公众号：**嵌入式校招菌**，对话框回复：**加群**